

智能体系统的自动化设计

Shengran Hu^{1,2}, Cong Lu^{1,2}, Jeff Clune^{1,2,3}

¹University of British Columbia, ²Vector Institute, ³Canada CIFAR AI Chair
{srhu, conglu}@cs.ubc.ca, jclune@gmail.com

摘要

研究人员正投入大量精力开发强大的通用智能体，其中基础模型（**Foundation Models**）被用作智能体系统（如思维链、自我反思、Toolformer）中的模块。然而，机器学习的历史告诉我们，手工设计的解决方案最终会被学习得到的解决方案所取代。本文描述了一个新兴的研究领域——智能体系统的自动化设计（**Automated Design of Agentic Systems, ADAS**），其目标是自动创建强大的智能体系统设计，包括发明新颖的构建模块和/或以新的方式组合它们。我们进一步证明，在 **ADAS** 中存在一种尚未探索但极具前景的方法：智能体可以用代码定义，并且可以通过一个元智能体在代码中编程出更好的智能体来自动发现新智能体。鉴于大多数编程语言是图灵完备的，这种方法理论上能够学习任何可能的智能体系统：包括新颖的提示、工具使用、工作流及其组合。我们提出了一种简单而有效的算法，称为元智能体搜索（**Meta Agent Search**），以展示这一思想：元智能体基于不断增长的先前发现档案，迭代地编程出有趣的新智能体。通过在编码、科学和数学等多个领域的大量实验，我们表明该算法能够逐步发明具有新颖设计的智能体，其性能大幅超越最先进的手工设计智能体。重要的是，我们一致观察到令人惊讶的结果：由元智能体搜索发明的智能体即使在跨领域和跨模型迁移时也能保持卓越性能，证明了它们的鲁棒性和通用性。只要我们能安全地开发它，我们的工作就展示了一个激动人心的新研究方向的潜力，即自动设计日益强大的智能体系统以造福人类。所有代码已在 <https://github.com/ShengranHu/ADAS> 开源。

1 引言

基础模型（Foundation Models, FMs）如 GPT（OpenAI, 2024; 2022）和 Claude（Anthropic, 2024b）正迅速被采用为强大的通用智能体，用于需要灵活推理和规划的智能体任务（Wang 等, 2024）。尽管基础模型近期取得了进展，但可靠地解决问题通常需要智能体是一个由多个组件构成的复合智能体系统，而非单一模型查询（Zaharia 等, 2024; Rockett & Schel, 2024）。此外，为了使智能体能够解决复杂的现实世界任务，它们通常需要访问外部工具，如搜索引擎、代码执行和数据库查询。因此，许多有效的智能体系统构建模块已被提出，例如思维链规划与推理（Wei 等, 2022; Yao 等, 2023; Hu & Clune, 2024）、记忆结构（Zhang 等, 2024c; Lewis 等, 2020）、工具使用（Schick 等, 2023; Qu 等, 2024）以及自我反思（Madaan 等, 2024; Shinn 等, 2023）。尽管这些智能体已在各种应用中取得了显著成功（Wang 等, 2024），但开发这些构建模块并将其组合成复杂的智能体系统通常需要领域特定的手动调整以及研究人员和工程师的大量努力。

然而，机器学习的历史揭示了一个反复出现的主题：随着我们获得更多的计算能力和数据（Sutton, 2019），手工创建的工件会逐渐被学习到的、更高效的解决方案所取代（Clune, 2019）。一个早期的例子来自计算机视觉领域，手工设计的特征如 HOG（Dalal & Triggs, 2005）最终被卷积神经网络（Convolutional Neural Networks, CNNs, Krizhevsky 等（2012））学习到的特征所取代。最近，自动机器学习（AutoML）方法（Hutter 等,

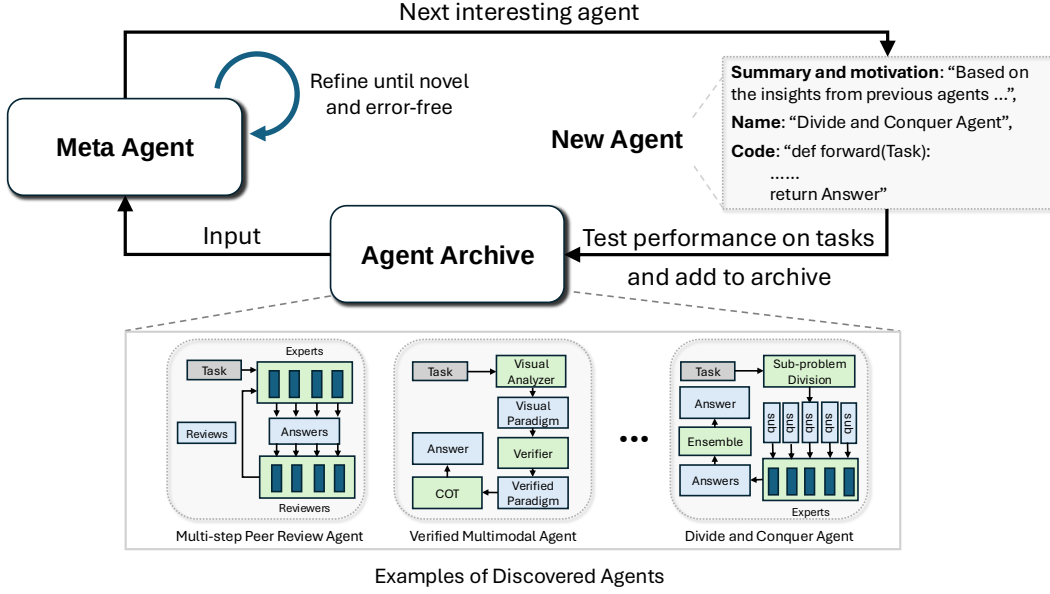


图 1：所提算法元智能体搜索的概述以及发现的智能体示例。在我们的算法中，我们指示“元”智能体迭代地编程新智能体，在任务上测试它们的性能，将它们添加到已发现智能体的存档中，并利用该存档在后续迭代中为元智能体提供信息。我们展示了运行中的三个示例智能体，所有名称均由元智能体生成。示例智能体的详细代码见附录 G。

2019 年) 以及人工智能生成算法 (AI-Generating Algorithms, 简称 AI-GAs, Clune (2019 年)) 也证明了学习得到的人工智能系统相较于手工设计的人工智能系统具有优越性。例如, 当前性能最佳的卷积神经网络模型来自神经架构搜索 (Neural Architecture Search, Elsken 等人, 2019 年; Shen 等人, 2023 年), 而非手工设计; 在大语言模型对齐中, 学习得到的损失函数 (Lu 等人, 2024a) 优于大多数手工设计的损失函数, 如 DPO (Rafailov 等人, 2024 年); 人工智能科学家 (The AI Scientist, Lu 等人, 2024b) 展示了一个自动化的研究流程, 包括开发新颖的机器学习算法; 此外, 在 OMNI-EPIC (Faldor 等人, 2024 年) 等工作中, 可以自动生成无穷无尽的机器人学习环境, 这些环境展现出惊人的创造力, 并且比手工方法更高效地创建环境 (更多示例见第 5 节)。因此, 本文提出一个新的研究问题: 我们能否自动化设计智能体系统?

为了探索上述研究问题, 我们描述了一个新形成的研究领域, 称之为智能体系统自动设计 (**Automated Design of Agentic Systems**, 简称 **ADAS**), 其目标是自动发明新颖的构建模块并设计强大的智能体系统 (第 2 节)。我们认为 **ADAS** 可能是开发强大智能体的最快路径, 并展示了初步证据表明学习得到的智能体可以显著优于手工设计的智能体。考虑到智能体系统中还有大量待发现的构建模块 (第 5 节), 我们的研究界需要很长时间才能将它们全部发现。即使我们成功发现了大多数有用的构建模块, 将它们组合成适用于大规模实际应用的有效智能体系统仍然具有挑战性且耗时, 因为构建模块之间有许多不同的组合和交互方式。相比之下, 借助 **ADAS**, 构建模块和智能体可以以自动化的方式学习得到。**ADAS** 不仅可能节省开发强大智能体的人力, 而且可能比手工设计更快地获得更有效的解决方案。

尽管少数现有工作可被视为自动设计智能体系统 (ADAS) 方法, 但其中大多数仅专注于设计提示 (Yang et al., 2024; Fernando et al., 2024), 极大地限制了它们在智能体中发明灵活设计模式的能力 (第 5 节)。本文表明, 存在一种尚未探索但极具前景的 ADAS 方法: 我们可以在代码中定义整个智能体系统, 并由一个“元”智能体在代码中编程出更优的智能体来自动发现新智能体。鉴于大多数编程语言 (如本文使用的 Python) 是图灵完备的 (Boyer & Moore, 1983; Ladha, 2024), 在代码空间内搜索理论上使 ADAS 算法能够发现任何可能的智能体系统, 包括所有组件, 例如

提示、工具使用、工作流等。此外，随着近期基础模型（FM）在编码方面日益熟练，我们可以利用 FM 作为元智能体，在代码中为 ADAS 创建新智能体，从而以自动化方式编程出新颖的智能体。

遵循上述思路，本文提出元智能体搜索（Meta Agent Search），作为 ADAS 中首批实现代码空间完整设计的算法之一（图 1）。元智能体搜索的核心概念是指示元智能体迭代地创建有趣的新智能体，对其进行评估，将其添加到存储已发现智能体的档案中，并利用该档案帮助元智能体在后续迭代中创建更有趣的新智能体。与利用人类趣味概念的现有开放式算法类似（Zhang et al., 2024a; Lu et al., 2024c），我们鼓励元智能体探索有趣的（例如新颖的或有价值的）智能体。为验证所提方法，我们在以下方面评估了所提出的元智能体搜索：（1）具有挑战性的 ARC 逻辑谜题任务（Chollet, 2019），旨在测试人工智能系统的通用智能；（2）四个流行的基准，涵盖阅读理解、数学、科学问题和多任务问题解决；（3）已发现智能体对留出领域和模型的可迁移性（第 4 节）。

我们的实验表明，所发现的智能体显著优于最先进的手工设计基线。例如，在 DROP（Dua 等人，2019）的阅读理解任务上，我们的智能体将 F1 分数提高了 **13.6/100**；在 MGSM（Shi 等人，2023）的数学任务上，准确率提高了 **14.4%**。此外，在跨领域迁移后，它们在 GSM8K（Cobbe 等人，2021）和 GSM-Hard（Gao 等人，2023）数学任务上的准确率分别比基线提高了 **25.9%** 和 **13.2%**。我们的算法相对于手工设计方案的优异性能，展示了 ADAS 在自动化设计智能体系统方面的潜力。此外，实验表明，所发现的智能体不仅在相似领域间迁移时表现良好，在跨不相似领域（如从数学到阅读理解）迁移时也展现出强大的性能。这凸显了元智能体搜索所发现的智能体系统的鲁棒性和可迁移性。总之，我们的工作开辟了许多令人兴奋的研究方向，并鼓励进一步的研究（第 6 节）。

2 智能体系统的自动化设计（ADAS）

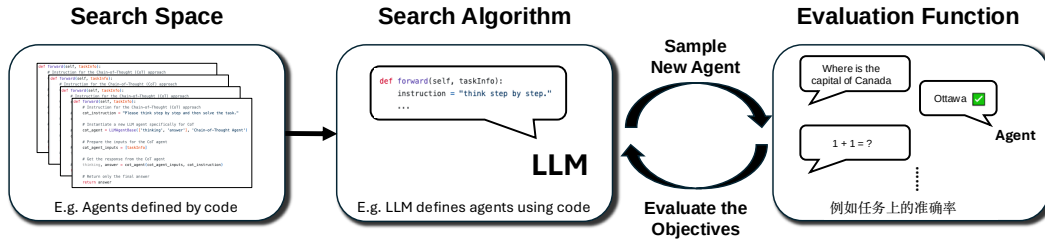


图 2：智能体系统自动化设计（ADAS）的三个关键组成部分。搜索空间决定了 ADAS 中可表示的智能体系统。搜索算法指定了 ADAS 方法如何探索搜索空间。评估函数定义了如何根据性能等目标来评估候选智能体。

在撰写本文时，社区尚未就智能体的定义或术语达成共识。这里，我们所说的智能体是指以基础模型（Foundation Models, FMs）作为 workflow 中的模块，通过规划、使用工具以及执行多个迭代处理步骤来解决问题的智能体系统（Chase, 2024; Ng, 2024）。本文描述了一个新兴的研究领域——智能体系统的自动化设计（ADAS）。与 AI-GAs（Clune, 2019）和 AutoML（Hutter 等人，2019）中的研究领域（如神经架构搜索（Elsken 等人，2019））类似，我们将 ADAS 形式化为一个优化过程，并确定了 ADAS 算法的三个关键组成部分（图 2）。

Formulation

智能体系统的自动化设计（ADAS）涉及使用搜索算法在搜索空间中搜索能够优化评估函数的智能体系统。

- **搜索空间：**搜索空间定义了哪些智能体系统可以被表示，从而在 ADAS 中被发现。例如，像 PromptBreeder (Fernando 等人, 2024) 这样的工作仅对智能体的文本提示进行变异，但其工作流等其他组件保持不变。因此，在这些搜索空间中，无法表示与预定义工作流不同的智能体。现有工作还探索了如图结构 (Zhuge 等人, 2024) 和前馈网络 (Liu 等人, 2023) 等搜索空间。
- **搜索算法：**搜索算法定义了 ADAS 算法如何探索搜索空间。由于搜索空间通常非常大甚至无界，应考虑探索与利用的权衡 (Sutton & Barto, 2018)。理想情况下，算法既能快速发现高性能的智能体系统，又能避免陷入局部最优。现有方法包括使用强化学习 (Zhuge 等人, 2024) 或由基础模型迭代生成新解决方案 (Fernando 等人, 2024) 作为搜索算法。
- **评估函数：**根据 ADAS 算法的应用场景，我们可能考虑不同的优化目标，如智能体的性能、成本、延迟或安全性。评估函数定义了如何在这些目标上评估候选智能体。例如，为了评估智能体在未见未来数据上的性能，一种简单的方法是计算任务验证数据上的准确率，这在现有工作中被普遍采用 (Zhuge 等人, 2024; Fernando 等人, 2024)。

尽管存在多种可能的搜索空间设计，且部分已得到探索（第 5 节），但仍有一种尚未开发却极具前景的方法：我们可以在代码中定义整个智能体系统，并由元智能体通过编程不断发现更优的新智能体。在代码空间内进行搜索，理论上使 ADAS 算法能够发现任何可能的构建模块（如提示、工具使用、工作流）以及以任意方式组合这些模块的智能体系统。该方法还为智能体设计模式提供了更好的可解释性，因为程序代码通常可读，从而简化调试并增强人工智能安全性。此外，与使用网络 (Liu 等, 2023) 或图 (Zhuge 等, 2024) 的搜索空间相比，在代码空间中搜索使我们能够更轻松地将基于现有的人类成果进行构建。例如，可以在 LangChain (LangChainAI, 2022) 等开源智能体框架内进行搜索，并基于所有现有构建模块（如 RAG、搜索引擎工具）进行开发。最后，由于基础模型擅长编码，利用代码搜索空间使我们能够在搜索过程中利用基础模型的现有专业知识。相比之下，在自定义搜索空间（如图）中的搜索算法可能因缺乏这些先验知识而效率低得多。因此，我们认为在 ADAS 中应更深入地研究将编程语言作为搜索空间的方法。

3 我们的算法：元智能体搜索

在本节中，我们提出元代理搜索 (Meta Agent Search)，这是一种简单而有效的算法，用于演示在代码中定义和搜索代理的方法。元代理搜索的核心思想是采用基础模型 (FM) 作为元代理，基于不断增长的先前发现档案，迭代地编程出有趣的新代理。虽然理论上任何可能的构建模块和代理系统都可以由元代理从头编程，但在实践中，如果不为元代理提供任何基本功能（如基础模型查询应用程序编程接口或现有工具），效率会很低。因此，在本文中，我们为元代理定义了一个简单的框架（100 行代码以内），为其提供一组基本必要功能，如查询基础模型或格式化提示。结果，元代理只需编程一个“前向”函数来定义新的代理系统，类似于趣味搜索 (FunSearch) (Romera-Paredes 等人, 2024) 中的做法。该函数接收任务信息，并输出代理对任务的响应。框架代码的细节以及使用该框架定义的代理示例可在附录 C 中找到。

如图 1 所示，元代理搜索的核心思想是让元代理在代码中迭代地编程新代理。算法流程如下：（1）档案（可选）用基线代理初始化，如思维链 (Chain-of-Thought) (Wei 等人, 2022) 和自我精炼 (Self-Refine) (Madaan 等人, 2024; Shinn 等人, 2023)。（2）以档案为条件，元代理通过生成代理系统新想法的高层描述，然后在代码中实现它，来设计一个新代理。该设计随后经过元代理的两步自我反思 (Madaan 等人, 2024; Shinn 等人, 2023)，以确保其新颖性。（3）使用目标领域的验证数据评估生成的代理。如果评估过程中出现错误，元代理执行一步自我反思以

改进设计，如有必要，此过程最多重复五次。（4）最后，该代理连同其评估度量一起被添加到档案中，并使用更新后的档案继续该过程，直到达到最大迭代次数。算法的伪代码在附录 H 中提供。

与现有利用人类趣味概念的开放式算法（Zhang et al., 2024a; Lu et al., 2024c）类似，我们鼓励元代理基于不断增长的先前发现档案，探索有趣的新颖（例如，新颖或有价值的）代理。在此，我们计算性能（例如，成功率或 F1 分数）作为元代理最大化的度量。提示词及更多细节见附录 B。

4 实验

我们进行了广泛的实验：（1）ARC 挑战（Chollet, 2019）（第 4.1 节），（2）四个流行的基准，评估代理在阅读理解、数学、科学问题和多任务问题解决方面的能力（第 4.2 节），以及（3）在数学上发现的代理对留出数学任务和非数学任务的可迁移性（第 4.3 节）。我们在不同任务中使用相同的算法实现，唯一的变化是提示词中包含的任务特定描述文本（详情见附录 B）。在所有实验中，我们发现发现的代理显著优于基线最先进的手工设计代理，并且在跨领域和跨模型迁移时仍保持优越性能。

4.1 案例研究：ARC 挑战

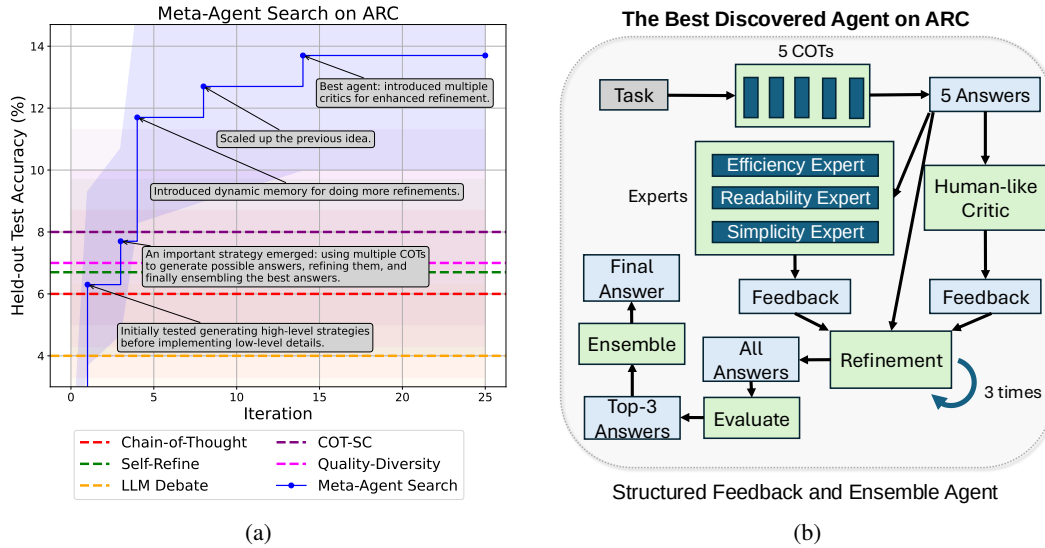


图 3：元代理搜索在 ARC 挑战上的结果。（a）元代理搜索基于不断增长的先前发现档案，逐步发现高性能代理。我们报告在留出测试集上评估代理五次的中位准确率和 95% 自助置信区间。（b）元代理搜索在 ARC 挑战上发现的最佳代理的可视化。该代理的详细实现见附录 D。

我们首先展示元代理搜索如何发现新颖的代理系统，并在抽象与推理语料库（ARC）挑战中超越现有最先进的手工设计代理。该挑战旨在通过 AI 系统获取新技能的能力来评估其通用智能。ARC 中的问题包括：（1）展示多个视觉输入输出网格模式的示例；（2）AI 系统从示例中学习网格模式的变换规则；（3）给定测试输入网格模式，预测输出网格模式。由于 ARC 中的每个问题都有独特的变换规则，这要求 AI 系统能够利用少量示例高效学习，并运用计数、几何和拓扑等能力。

设置。遵循常见做法，我们要求代理编写变换规则的代码，而不是直接回答。我们在框架中提供工具函数（在第 3 节中描述），用于评估生成的变换代码。鉴于 ARC 对当前 AI 系统构成的重大挑战，我们从“公共训练集（简单）”中网格维度为 $\leq 5 \times 5$ 的问题中采样数据。我们分别采样一个包含 20 个问题的验证集和一个包含 60 个问题的测试集，用于搜索和测试。我们通过让代理在验证集和测试集上各评估五次来计算其验证准确率和测试准确率，以减少基础模型随机采样带来的方差。我们在留出的测试集上评估所有发现的代理，并在图 3 中报告测试准确率。元代理搜索运行 25 次迭代，元代理使用 GPT-4，而发现的代理和基线使用 GPT-3.5 进行评估以降低计算成本。更多算法细节和 ARC 问题示例见附录 D。

基线。我们将所提方法与五种最先进的手工设计智能体进行了比较：（1）思维链（Chain-of-Thought, COT, Wei 等人, 2022），该方法指示智能体在回答前输出推理过程，通过中间步骤提升复杂问题求解能力；（2）带思维链的自洽性（Self-Consistency with Chain-of-Thought, COT-SC, Wang 等人, 2023b），该方法对思维链产生的多个并行答案进行集成，以生成更准确的答案；（3）自我精炼（Self-Refine, Madaan 等人, 2024; Shinn 等人, 2023），该方法允许迭代式自我反思，以纠正先前尝试中的错误；（4）大语言模型辩论（LLM-Debate, Du 等人, 2023），该方法使不同大语言模型相互辩论，利用多样化视角寻找更优答案；（5）质量多样性（Quality-Diversity），这是智能探索（Intelligent Go-Explore, Lu 等人, 2024c）的简化版本，通过生成并集成多样化答案来更好地探索潜在解决方案。所选基线代表了智能体文献中广泛采用的设计，体现了各类应用中频繁使用的关键设计模式与方法。此处“最先进”指这些基线设计体现了领域内的重要进展与实践。我们还将所有基线作为元智能体搜索（Meta Agent Search）档案库的初始种子，空初始化的额外结果见附录 I。为确保公平比较，所有基线实现均采用与元智能体相同的框架开发，提供了一致且公正的评估环境。关于基线的更多细节见附录 F。

结果与分析。如图 3a 所示，元代理搜索（Meta Agent Search）有效且渐进地发现了性能优于最先进手工设计基线的代理。重要突破在文本框中突出显示。正如先前关于开放性和人工智能遗传算法（AI-GAs）的研究（Zhang et al., 2024a; Faldor et al., 2024; Wang et al., 2019; 2020; Lehman & Stanley, 2011）所强调的关键，元代理搜索基于不断增长的先前踏脚石档案进行创新。例如，在第 3 次迭代中出现了一个重要的设计模式，它使用多个思维链（COTs）生成可能的答案，对其进行精炼，并最终集成最佳答案。这成为了后续设计倾向于利用的关键踏脚石。此外，图 3b 展示了发现的最佳代理，其中采用了一种复杂的反馈机制以更有效地精炼答案。仔细观察搜索过程可以发现，这种复杂的反馈机制并非突然出现。相反，在第 5、11 和 12 次迭代中分别出现了纳入多样化反馈、通过专家评估各种特定特征（如效率和简洁性）以及模拟类人反馈的想法。最终机制是基于这三个踏脚石的创新。这表明，尽管这些踏脚石在出现时并未立即取得高性能，但后续的发现通过结合不同的踏脚石从这些创新中受益，类似于通过大语言模型（LLMs）进行的进化中的交叉（Meyerson et al., 2023）。总体而言，结果展示了自动设计代理搜索（ADAS）的潜力以及元代理搜索的有效性，即通过踏脚石的创新与组合，渐进地发现性能优于最先进手工设计基线的代理，并发明新颖的设计模式。

4.2 推理与问题求解领域

实验设置。接下来，我们探究本文算法在数学、阅读和推理领域提升智能体能力的潜力。我们在四个常用基准上测试元智能体搜索：用于评估阅读理解 DROP（Dua 等, 2019）；用于评估多语言环境下数学能力的 MGSM（Shi 等, 2023）；用于评估多任务问题求解的 MMLU（Hendrycks 等, 2021）；以及用于评估解决科学领域高难度（研究生水平）问题能力的 GPQA（Rein 等, 2023）。搜索在每个领域内独立进行。元智能体搜索运行 30 次迭代。元智能体使用 GPT-

4 (OpenAI, 2024) , 而发现的智能体和基线使用 GPT-3.5 (OpenAI, 2022) 进行评估。数据集和实验设置的更多细节见附录 E。

基线。我们采用第 4.1 节介绍的所有基线。此外, 由于上述领域需要较强的推理能力, 我们额外加入两个专门增强智能体推理能力的基线, 以进行更全面的比较: 回退抽象 (Zheng 等, 2023), 指导智能体首先考虑解决问题所涉及的原理以改善推理; 角色分配 (Xu 等, 2023), 为基座模型分配不同角色以获得更好的答案。此外, 我们将本文方法与最先进的提示优化基线 OPRO (Yang 等, 2024) 进行比较, 以突出学习智能体所有可能组件而非仅关注提示的优势。基线的更多细节见附录 F。

表 1: 元智能体搜索与最先进的手工设计智能体在多个领域的性能比较。元智能体搜索在每个领域都发现了优于基线的智能体。我们报告了在留出测试集上的测试准确率和 95% 自助置信区间。搜索在每个领域独立进行。在此处及以下所有表格中, 我们将每个领域性能最高的条目加粗, 同时将所有中位数落在最高性能处理 95% 置信区间内的条目加粗。

Agent Name	F1 Score	Accuracy (%)		
	Reading Comprehension	Math	Multi-task	Science
State-of-the-art Hand-designed Agents				
Chain-of-Thought (Wei et al., 2022)	64.2 ± 0.9	28.0 ± 3.1	65.4 ± 3.3	29.2 ± 3.1
COT-SC (Wang et al., 2023b)	64.4 ± 0.8	28.2 ± 3.1	65.9 ± 3.2	30.5 ± 3.2
Self-Refine (Madaan et al., 2024)	59.2 ± 0.9	27.5 ± 3.1	63.5 ± 3.4	31.6 ± 3.2
LLM Debate (Du et al., 2023)	60.6 ± 0.9	39.0 ± 3.4	65.6 ± 3.3	31.4 ± 3.2
Step-back Abstraction (Zheng et al., 2023)	60.4 ± 1.0	31.1 ± 3.2	65.1 ± 3.3	26.9 ± 3.0
Quality-Diversity (Lu et al., 2024c)	61.8 ± 0.9	23.8 ± 3.0	65.1 ± 3.3	30.2 ± 3.1
Role Assignment (Xu et al., 2023)	65.8 ± 0.9	30.1 ± 3.2	64.5 ± 3.3	31.1 ± 3.1
Automated Design of Agentic Systems on Different Domains				
Prompt Optimization (Yang et al., 2024)	69.1 ± 0.9	30.6 ± 3.2	67.6 ± 3.2	32.9 ± 3.2
Meta Agent Search (Ours)	79.4 ± 0.8	53.4 ± 3.5	69.6 ± 3.2	34.6 ± 3.2

结果与分析。跨多个领域的结果表明, 元代理搜索 (Meta Agent Search) 能够发现优于最先进的手工设计代理 (表 1)。我们想强调在阅读理解和数学领域中, 学习到的代理与手工设计代理之间存在显著差距, F1 分数分别提高了 **13.6/100**, 准确率提高了 **14.4%**。虽然元代理搜索在多任务和科学领域也优于基线, 但差距较小。我们假设, 对于科学和多任务领域中的挑战性问题, 基础模型 (FMs) 中的知识不足以解决这些问题, 从而限制了通过优化代理系统带来的改进, 而这一问题将随着基础模型的改进而减弱。相比之下, 在阅读理解和数学领域, 基础模型具备足够的知识来解决问题, 错误可能主要是幻觉或计算错误, 这些可以通过精心设计的代理系统 (如元代理搜索所发现的系统) 来缓解。此外, 与提示优化方法相比, 结果表明我们提出的元代理搜索在所有领域都持续优于它们。这一比较进一步强化了我们的论点: 在代码中定义代理并允许学习所有组件具有显著优势。总体而言, 跨多个领域的结果展示了元代理搜索在为特定领域搜索定制代理方面的有效性。随着我们继续为各种应用创建代理 (Wang 等人, 2024), 这可能越来越有助于节省人力并开发更好的任务特定代理。

4.3 泛化性与可迁移性

在前面的章节中, 我们说明了元代理搜索可以为单个任务找到有效的代理。在本节中, 我们进一步展示所发现代理的可迁移性和泛化性。为了展示所发明构建块的泛化性以及

符号模式，我们将从 MGSM（数学）领域发现的智能体迁移到数学和非数学领域，以测试它们在不同任务上的泛化能力。我们评估 MGSM 中排名前 3 的智能体，将它们迁移到（1）流行的数学领域：GSM8K（Cobbe 等人，2021）、GSM-Hard（Gao 等人，2023），以及（2）非数学领域：MMLU（多任务）和 DROP（阅读理解），详见第 4.2 节。如表 2 所示，元智能体搜索始终优于基线。值得注意的是，在数学领域内迁移时，我们的智能体在 GSM8K 上准确率提高了 **25.9%**，在 GSM-Hard 上提高了 **13.2%**。更令人惊讶的是，我们发现数学领域发现的智能体也可以迁移到非数学领域。虽然它们的性能不能完全匹配为目标领域专门设计的智能体，但它们仍然优于最先进的手工设计基线。跨领域迁移的更多结果见附录 A。

我们还在 ARC 上跨不同基础模型迁移智能体时观察到类似的优越性。我们测试了在 ARC 上用 GPT-3.5 评估获得最佳测试准确率的前 3 个智能体，然后将它们迁移到 Claude-Haiku（Anthropic, 2024a）、GPT-4（OpenAI, 2024）和 Claude-Sonnet（Anthropic, 2024b）。如表 3 所示，我们观察到搜索到的智能体始终优于手工设计的智能体，且差距显著。值得注意的是，我们发现 Anthropic 最强大的模型 Claude-Sonnet 在所有测试模型中表现最佳，使我们最好的智能体在 ARC 上达到近 **50%** 的准确率。这些跨领域和跨模型的迁移结果凸显了元智能体搜索发现可泛化设计模式和智能体系统的能力。

表 2：将 MGSM（数学）中的顶级智能体迁移到留出的数学和非数学领域时的性能。 GSM8K 和 GSM-Hard 是留出的数学领域，MMLU 用于多任务，DROP 用于阅读理解。元智能体搜索发现的智能体在所有领域始终优于基线。我们报告测试准确率和 95% 自助置信区间。顶级智能体的名称由元智能体搜索生成。

智能体名称	准确率 (%)	F1 分数			
	MGSM	GSM8K	GSM-Hard	MMLU	DROP
Manually Designed Agents					
Chain-of-Thought (Wei et al., 2022)	28.0 ± 3.1	34.9 ± 3.2	15.0 ± 2.5	65.4 ± 3.3	64.2 ± 0.9
COT-SC (Wang et al., 2023b)	28.2 ± 3.1	37.8 ± 3.4	15.5 ± 2.5	65.9 ± 3.2	64.4 ± 0.8
Self-Refine (Madaan et al., 2024)	27.5 ± 3.1	38.9 ± 3.4	15.1 ± 2.4	63.5 ± 3.4	59.2 ± 0.9
LLM Debate (Du et al., 2023)	39.0 ± 3.4	43.6 ± 3.4	17.4 ± 2.6	65.6 ± 3.3	60.6 ± 0.9
Step-back Abstraction (Zheng et al., 2023)	31.1 ± 3.2	31.5 ± 3.3	12.2 ± 2.3	65.1 ± 3.3	60.4 ± 1.0
Quality-Diversity (Lu et al., 2024c)	23.8 ± 3.0	28.0 ± 3.1	14.1 ± 2.4	65.1 ± 3.1	61.8 ± 0.9
Role Assignment (Xu et al., 2023)	30.1 ± 3.2	37.0 ± 3.4	18.0 ± 2.7	64.5 ± 3.3	65.8 ± 0.9
Top Agents Searched on MGSM (Math)		Transferred within Math Domains		Transferred beyond Math Domains	
Dynamic Role-Playing Architecture	53.4 ± 3.5	69.5 ± 3.2	31.2 ± 3.2	62.4 ± 3.4	70.4 ± 0.9
Structured Multimodal Feedback Loop	50.2 ± 3.5	64.5 ± 3.4	30.1 ± 3.2	67.0 ± 3.2	70.4 ± 0.9
Interactive Multimodal Feedback Loop	47.4 ± 3.5	64.9 ± 3.3	27.6 ± 3.2	64.8 ± 3.3	71.9 ± 0.8

5 相关工作

智能体系统。研究人员为不同应用开发了各种构建模块和设计模式。智能体系统的重要构建模块包括：提示技术（Chen et al., 2023a; Schulhoff et al., 2024）、基于思维链的规划与推理方法（Wei et al., 2022; Yao et al., 2023; Hu & Clune, 2024）、反思（Madaan et al., 2024; Shinn et al., 2023）、在代码中为具身智能体开发新技能（Wang et al., 2023a; Vemprala et al., 2023）、外部记忆与检索增强生成（Zhang et al., 2024c; Lewis et al., 2020）、工具使用（Qu et al., 2024; Schick et al., 2023; Nakano et al., 2021）、为智能体系统中的基础模型模块分配不同角色并使其协作（Hong et al., 2023; Wu et al., 2023; Qian et al., 2023; Xu et al., 2023; Qian et al., 2024），以及使智能体能够自我指导下一步行动（Richards, 2023）等。尽管社区在开发上述所有重要技术方面投入了大量努力，但这只是已发现构建模块的部分列表，还有更多有待

表 3：将 GPT-3.5 的顶级智能体迁移到其他基础模型时在 ARC 上的性能。由元智能体搜索发现的智能体在不同模型上一致优于基线。我们报告了测试准确率和 95% 自助置信区间。顶级智能体的名称由元智能体搜索生成。† 我们手动更改了这个名称，因为原始生成的名称令人困惑。

Agent Name	Accuracy on ARC (%)			
	GPT-3.5	Claude-Haiku	GPT-4	Claude-Sonnet
Manually Designed Agents				
Chain-of-Thought (Wei et al., 2022)	6.0 ± 2.7	4.3 ± 2.2	17.7 ± 4.4	25.3 ± 5.0
COT-SC (Wang et al., 2023b)	8.0 ± 3.2	5.3 ± 2.5	19.7 ± 4.5	26.3 ± 4.9
LLM Debate (Du et al., 2023)	4.0 ± 2.2	1.7 ± 1.5	19.0 ± 4.5	24.7 ± 4.8
Self-Refine (Madaan et al., 2024)	6.7 ± 2.7	6.3 ± 2.8	23.0 ± 5.2	39.3 ± 5.5
Quality-Diversity (Lu et al., 2024c)	7.0 ± 2.9	3.3 ± 2.2	23.0 ± 4.7	31.7 ± 5.3
Top Agents Searched with GPT-3.5		Transferred to Other FMs		
Structured Feedback and Ensemble Agent	13.7 ± 3.9	5.0 ± 2.5	30.0 ± 5.2	38.7 ± 5.5
Hierarchical Committee Reinforcement Agent	13.3 ± 3.8	8.3 ± 3.2	32.3 ± 8.9	39.7 ± 5.5
Dynamic Memory and Refinement Agent [†]	12.7 ± 3.9	9.7 ± 3.3	37.0 ± 5.3	48.3 ± 5.7

发现。因此，在本文中，我们描述了一个新形成的研究领域——自动设计智能体系统（ADAS），旨在以自动化方式发明新颖的构建模块并设计强大的智能体系统。

AI 生成算法与自动机器学习。人工智能生成算法（AI-Generating Algorithms, AI-GAs, Clune, 2019）与自动机器学习（AutoML, Hutter 等, 2019）的研究旨在通过学习来替代人工智能系统中的手工设计组件。该领域有三大支柱：（1）元学习架构，（2）元学习学习算法，以及（3）生成学习环境与训练数据（Clune, 2019）。神经架构搜索（Neural Architecture Search, Elsken 等, 2019; Lu 等, 2019; Hu 等, 2021）通过自动化神经网络设计体现了第一支柱，而 MAML (Finn 等, 2017) 和元强化学习（Meta-RL, Wang 等, 2016; Duan 等, 2017; Norman 与 Clune, 2023; Zintgraf 等, 2021a; b）等工作则体现了第二支柱，专注于“学会学习”以提升样本效率与泛化能力。第三支柱包括 POET (Wang 等, 2019; Dharna 等, 2022; Wang 等, 2020) 和 OMNI-EPIC (Faldor 等, 2024) 等工作，它们以开放式方式生成学习环境。我们将智能体系统的自动化设计（Automated Design of Agentic Systems）定位在第一和第二支柱中：元学习智能体架构，并利用上下文学习（in-context learning）实现“学会学习”，如抽象推理语料库（ARC）挑战所示（第 4.1 节）。此外，近期人工智能生成算法与自动机器学习的进展也整合了基础模型（Foundation Models, FMs）来编写代码，如 FunSearch (Romera-Paredes 等, 2024) 和 EoH (Liu 等, 2024) 中，基础模型发现优化算法。在 DiscoPOP (Lu 等, 2024a) 中，基础模型为偏好学习编写损失函数，而 Eureka (Ma 等, 2023) 和语言到奖励（language-to-reward, Yu 等, 2023）使基础模型能够为强化学习编写奖励函数。OMNI-EPIC (Faldor 等, 2024) 允许基础模型创建机器人学习环境。类似地，我们使基础模型能够用代码编写新的智能体。

现有的智能体系统自动化设计尝试。有两类工作尝试实现自动化智能体系统设计（ADAS）：一类专注于学习更优的提示，另一类则学习提示之外的更多组件。大多数工作属于第一类，其中基础模型（FM）被用于自动化提示工程，主要通过改进指令的措辞来提升推理能力（Yang et al., 2024; Fernando et al., 2024; Zhou et al., 2024a; Yuksekgonul et al., 2024）。然而，这些提示往往具有领域特定性，难以泛化。一些工作优化提示中的角色定义（Yuan et al., 2024; Chen et al., 2023c;b; Wu et al., 2023），因为为智能体分配人格或角色已被证明是有益的（Xu et al., 2023）。尽管调整提示可以提升性能，但其他组件保持不变，限制了可发现的智能体空间。第二类工作探索较少，涉及学习额外组件，如工作流，通常将智能体表示为网络或图。在这些表述中，带有特定提示的基础模型被视为节点上文本的变换函数，文本的信息流被视为边。例如，DyLAN (Liu et al., 2023) 使用基础模型优化网络中节点之间的连接，DSPy (Khattab et al., 2024) 和 Trace (Cheng et al., 2024) 在一组可能节点的笛卡尔积上进行优化，GPT-Swarm (Zhuge et al., 2024) 使用强化学习优化节点连接。尽管这些方法优化了工作流，但许多组件

如工具使用保持固定。智能体优化器 (AgentOptimizer) (Zhang 等, 2024b) 学习智能体中使用的工具, 自动流 (AutoFlow) (Li 等, 2024) 提出一种新语言来优化工作流, 智能体符号学习 (Agent Symbolic Learning) (Zhou 等, 2024b) 尝试同时学习提示、工具和工作流。虽然这些工作具有相似的动机, 即学习智能体中的更多组件, 但它们要么未能覆盖智能体系统中所有可能的设计, 要么为搜索算法提供了更困难的搜索空间。相比之下, 本文的工作将所有组件用代码表示, 允许智能体系统中所有可能的设计, 并为基础模型 (FM) 引导的搜索提供了一个有前景的搜索空间, 因为编码任务是基础模型训练中最重要的任务之一。

6 讨论与结论

安全注意事项。尽管在我们当前使用的基础模型 (Foundation Models, FMs) 的设置下, 模型生成的代码极不可能执行明显恶意的行为, 但由于模型能力或对齐 (alignment) 的局限性, 此类代码仍可能产生破坏性作用 (Rokon et al., 2020; Chen et al., 2021)。为应对这些风险, 我们已实施安全措施, 包括在安全、隔离的环境中对所有生成的代码进行容器化执行, 进行彻底的人工检查以验证不存在有害行为, 并在我们的代码库中给出明确警告以提醒用户潜在风险。这些做法与文献中既定的安全标准一致, 例如 SWE-Bench (Jimenez et al., 2024) 和 Voyager (Wang et al., 2023a) 中的标准, 它们同样优先考虑受控的执行环境。

所提出的智能体系统自动设计 (Automated Design of Agentic Systems, ADAS) 在人工智能生成算法 (AI-GA) 研究中开辟了一个新领域, 有可能超越当前的人工方法, 加速通用人工智能 (Artificial General Intelligence, AGI) 的发展 (Clune, 2019)。这引发了关于提升人工智能能力的更广泛问题, 该主题在先前的工作中已被广泛讨论 (Clune, 2019; Ecoffet et al., 2020; Bostrom, 2002; Yudkowsky et al., 2008; Bengio et al., 2024), 但超出了本文的范围。我们认为发表这项工作总体上是利大于弊的。它揭示了强大的 ADAS 算法可以通过对基础模型的应用程序编程接口 (API) 访问轻松编程实现, 无需昂贵的硬件 (如 GPU), 从而让社区了解其可获取性和启示。此外, ADAS 可以通过自动化设计显式、可解释的工作流来增强智能体系统的安全性, 通过更高的可控性和可审计性降低恶意行为的风险。

我们认为, 鉴于智能体系统在现实世界应用中的日益普及 (图罗, 2024 年), 关于 ADAS 及其安全影响的讨论是及时的。ADAS 能够简化安全可靠智能体的创建过程, 从而增强人工智能在健康和经济等领域造福人类的潜力 (阿莫迪, 2024 年)。此外, 随着自我改进型人工智能系统日益突出 (克卢恩, 2019 年; 费尔南多等人, 2024 年; 卢等人, 2024a; 泽利克曼等人, 2022 年), 其持续发展似乎不可避免。通过分享这项工作, 我们旨在激发对安全 ADAS 算法的进一步研究——可能纳入宪法式 AI (Constitutional AI) 等机制 (白等人, 2022 年)——以确保 AI-GA 和自我改进型人工智能的进步能够产生既强大又与人类价值观一致的智能系统, 最终促进更安全的人工智能发展。

未来工作。我们的工作还开辟了许多未来的研究方向:

- **高阶 ADAS。**由于 ADAS 中用于在代码中编程新智能体的元智能体本身也是一个智能体, ADAS 可以变得自指涉, 即元智能体也可以通过 ADAS 得到改进。实现更高阶的元学习, 以允许元智能体乃至元元智能体等的学习, 将是一个令人兴奋的方向 (卢等人, 2023 年; 施米德胡贝, 1987 年; 2003 年; 泽利克曼等人, 2024 年)。
- **在线持续学习。**随着智能体的部署, 它们将从任务环境和用户那里接收到大量反馈。基于这些广泛反馈持续改进智能体对人类开发者来说具有挑战性。然而, 借助 ADAS 自动化设计和增强智能体, 在线持续学习在部署后变得可行。
- **多目标 ADAS。**本文仅考虑单一目标 (即性能) 进行优化, 但在实践中, 通常需要考虑多个目标, 例如智能体系统的成本、延迟和鲁棒性 (Hu et al., 2021; Huang et al., 2023)。因此, 将多目标搜索算法 (Deb et al., 2002) 集成到 ADAS 中可能具有前景。
- **更好地理解基础模型。**来自神经架构搜索 (Huang et al., 2023) 的研究表明, 通过观察涌现的架构, 我们可以获得更多关于

神经网络的洞见。本文也从结果中获得了关于基础模型的洞见。例如，使用 GPT-3.5 的最佳智能体涉及复杂的反馈机制，但当我们迁移到其他先进模型时，具有更简单反馈机制但更多精炼步骤的智能体成为更好的智能体（第 4.3 节）。这表明 GPT-3.5 在评估和精炼答案方面的能力可能较弱，因此需要复杂的反馈机制来实现更好的精炼，而其他先进模型则从更简单的反馈机制中获益更多。

- **更复杂的领域。**目前，本文仅在单步问答任务上评估元智能体搜索。将该方法扩展到更复杂的领域将是有趣的，例如涉及与复杂环境进行多步交互的真实世界应用。
- **用更多现有构建模块为 ADAS 提供种子。**尽管理论上我们可以允许智能体系统中的任何组件在代码空间中从头编程，但在实践中效率不高。因此，站在现有人类努力的肩膀上探索 ADAS 将是有趣的，例如搜索引擎工具、检索增强生成（Lewis et al., 2020），或来自现有智能体框架（如 LangChain（LangChainAI, 2022））的函数。此外，支持基础模型中的多模态能力（例如视觉）或允许智能体系统中使用不同的基础模型也是有趣的。这将使元智能体能够根据指令的难度以及数据隐私是否优先，灵活地从不同的基础模型中进行选择。
- **新颖性搜索算法。**在元代理搜索中，搜索算法的设计相对简单，仅专注于探索有趣的新设计。更精细的搜索算法设计可以是一个有前景的未来方向。例如，可以融入来自质量多样性（Quality-Diversity）（Mouret & Clune, 2015; Cully & Demiris, 2017）、人工智能生成（Clune, 2019）和开放式算法（Faldor et al., 2024; Zhang et al., 2024a; Stanley & Lehman, 2015; Stanley et al., 2019）的更复杂思想。也可以纳入更经典的方法来平衡探索与利用（Sutton & Barto, 2018; Liu et al., 2024）。
- **更智能的评估函数。**在本文中，我们仅在评估集上评估发现的代理，并使用数值性能结果。然而，这种方法既昂贵又丢失了大量信息。一个有前景的未来方向是使元代理能够在评估过程中分析详细的运行日志，这些日志包含丰富的失败和成功模式信息，有助于更好地调试和改进代理系统（Zhou et al., 2024b）。此外，许多任务涉及主观答案评估（Chiang et al., 2024; Lu et al., 2024b），没有标准答案。在 ADAS 中设计新颖的评估函数以解决这些任务也很重要。最后，在本文中，我们在搜索过程中仅针对一个领域。探索 ADAS 算法在专门搜索能够在多个领域表现良好的代理时，是否能设计出更好的通用代理，将是一个有趣的方向。
- **理解人类组织中复杂性的涌现。**除了可能节省研究者的精力并改进智能体系统的手工设计之外，自动化智能体系统设计（ADAS）的研究在科学上也颇具吸引力，因为它揭示了人类组织与社会中复杂性涌现的起源。智能体系统是一种主要基于自然语言运行的机器学习系统——这种表示形式对人类可解释，并且人类在构建组织与社会时也使用它。因此，智能体系统与人类组织之间存在紧密联系，正如将人类公司的组织结构融入智能体的工作（Hong 等，2023）或用智能体模拟人类小镇的工作（Park 等，2023）所示。因此，ADAS 的研究或许能让我们观察如何创建一组简单的条件，并让算法从简单性中自我引导，从而在类似于人类社会的系统中产生复杂性。

结论。本文提出了一个新的研究问题——自动化智能体系统设计（ADAS），旨在自动发明新颖的构建模块并设计强大的智能体系统。我们证明了一种有前景的 ADAS 方法是在代码中定义智能体，从而允许一个“元”智能体通过代码编程自动发现新的智能体。基于这一思想，我们提出了元智能体搜索，其中元智能体迭代地基于先前的发现来编程有趣的新智能体。实验表明，元智能体搜索在大量领域中始终优于最先进的手工设计智能体，并且所发现的智能体在不同模型和领域之间具有良好的迁移性。总体而言，我们的工作展示了一个令人兴奋的新研究方向——自底向上地全面自动化开发强大智能体系统的潜力。

致谢

本研究得到了 Vector Institute、加拿大 CIFAR AI 主席计划、Schmidt Futures 和 Open Philanthropy 的资助、NSERC 发现基金以及 Rafael Cosman 的慷慨捐赠的支持。我们感谢 Jenny Zhang、Rach Pradhan、Ruiyu Gou、Nicholas Ioannidis 和 Eunjeong Hwang 的深刻讨论与反馈。

参考文献

- Dario Amodei. Machines of loving grace, October 2024. URL <https://darioamodei.com/machines-of-loving-grace>.
- Anthropic. Introducing the next generation of claude. <https://www.anthropic.com/news/claude-3-family>, March 2024a. Blog post.
- Anthropic. Introducing claude 3.5 sonnet. <https://www.anthropic.com/news/claude-3-5-sonnet>, June 2024b. Blog post.
- Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073*, 2022.
- Yoshua Bengio, Geoffrey Hinton, Andrew Yao, Dawn Song, Pieter Abbeel, Trevor Darrell, Yuval Noah Harari, Ya-Qin Zhang, Lan Xue, Shai Shalev-Shwartz, et al. Managing extreme ai risks amid rapid progress. *Science*, 384(6698):842–845, 2024.
- N Bostrom. Existential Risks: analyzing human extinction scenarios and related hazards. *Journal of Evolution and Technology*, 9, 2002.
- Robert S Boyer and J Strother Moore. *A mechanical proof of the Turing completeness of pure LISP*. CiteSeer, 1983.
- Harrison Chase. What is an agent? <https://blog.langchain.dev/what-is-an-agent/>, June 2024. Blog post.
- Banghao Chen, Zhaofeng Zhang, Nicolas Langrené, and Shengxin Zhu. Unleashing the potential of prompt engineering in large language models: a comprehensive review. *arXiv preprint arXiv:2310.14735*, 2023a.
- Guangyao Chen, Siwei Dong, Yu Shu, Ge Zhang, Sesay Jaward, Karlsson Börje, Jie Fu, and Yemin Shi. Autoagents: The automatic agents generation framework. *arXiv preprint*, 2023b.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, et al. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors. In *The Twelfth International Conference on Learning Representations*, 2023c.
- Ching-An Cheng, Allen Nie, and Adith Swaminathan. Trace is the next autodiff: Generative optimization with rich feedback, execution traces, and llms. *Advances in Neural Information Processing Systems*, 37:71596–71642, 2024.
- Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios Nikolas Angelopoulos, Tianle Li, Dacheng Li, Hao Zhang, Banghua Zhu, Michael Jordan, Joseph E. Gonzalez, and Ion Stoica. Chatbot arena: An open platform for evaluating llms by human preference, 2024.
- François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.
- Jeff Clune. Ai-gas: Ai-generating algorithms, an alternate paradigm for producing general artificial intelligence. *arXiv preprint arXiv:1905.10985*, 2019.

- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Antoine Cully and Yiannis Demiris. Quality and diversity optimization: A unifying modular framework. *IEEE Transactions on Evolutionary Computation*, 22(2):245–259, 2017.
- N. Dalal and B. Triggs. Histograms of oriented gradients for human detection. In *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR’05)*, volume 1, pp. 886–893 vol. 1, 2005. doi: 10.1109/CVPR.2005.177.
- Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and TAMT Meyarivan. A fast and elitist multiobjective genetic algorithm: Nsga-ii. *IEEE transactions on evolutionary computation*, 6(2): 182–197, 2002.
- Aaron Dharna, Amy K Hoover, Julian Togelius, and Lisa B Soros. Transfer dynamics in emergent evolutionary curricula. *IEEE Transactions on Games*, 15(2):157–170, 2022.
- Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325*, 2023.
- Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In Jill Burstein, Christy Doran, and Thamar Solorio (eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 2368–2378, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1246.
- Yan Duan, John Schulman, Xi Chen, Peter L. Bartlett, Ilya Sutskever, and Pieter Abbeel. RL²: Fast reinforcement learning via slow reinforcement learning. In *International Conference on Learning Representations*, 2017.
- Adrien Ecoffet, Jeff Clune, and Joel Lehman. Open questions in creating safe open-ended AI: Tensions between control and creativity. In *Conference on Artificial Life*, pp. 27–35. MIT Press, 2020.
- Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. Neural architecture search: A survey. *Journal of Machine Learning Research*, 20(55):1–21, 2019.
- Maxence Faldor, Jenny Zhang, Antoine Cully, and Jeff Clune. Omni-epic: Open-endedness via models of human notions of interestingness with environments programmed in code. *arXiv preprint arXiv:2405.15568*, 2024.
- Chrisantha Fernando, Dylan Sunil Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution, 2024.
- Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *International conference on machine learning*, pp. 1126–1135. PMLR, 2017.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models. In *International Conference on Machine Learning*, pp. 10764–10799. PMLR, 2023.
- Ryan Greenblatt. Getting 50% sota on arc-agi with gpt-4. <https://redwoodresearch.substack.com/p/getting-50-sota-on-arc-agi-with-gpt>, July 2024. Technical Report.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021.

- Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, et al. Metagpt: Meta programming for multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023.
- Shengran Hu and Jeff Clune. Thought Cloning: Learning to think while acting by imitating human thinking. *Advances in Neural Information Processing Systems*, 36, 2024.
- Shengran Hu, Ran Cheng, Cheng He, Zhichao Lu, Jing Wang, and Miao Zhang. Accelerating multi-objective neural architecture search by random-weight evaluation. *Complex & Intelligent Systems*, pp. 1–10, 2021.
- Shihua Huang, Zhichao Lu, Kalyanmoy Deb, and Vishnu Naresh Boddeti. Revisiting residual networks for adversarial robustness. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 8202–8211, 2023.
- Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren. *Automated machine learning: methods, systems, challenges*. Springer Nature, 2019.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQm66>.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, Heather Miller, et al. Dspy: Compiling declarative language model calls into state-of-the-art pipelines. In *The Twelfth International Conference on Learning Representations*, 2024.
- Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012.
- Abraham Ladha. Lecture 11: Turing-completeness. <https://faculty.cc.gatech.edu/~ladha/S24/4510/L11.pdf>, 2024. CS 4510 Automata and Complexity, February 21st, 2024, Scribed by Rishabh Singhal.
- LangChainAI. Langchain: Build context-aware reasoning applications. <https://github.com/langchain-ai/langchain>, 2022.
- Joel Lehman and Kenneth O Stanley. Abandoning objectives: Evolution through the search for novelty alone. *Evolutionary computation*, 19(2):189–223, 2011.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33: 9459–9474, 2020.
- Zelong Li, Shuyuan Xu, Kai Mei, Wenyue Hua, Balaji Rama, Om Raheja, Hao Wang, He Zhu, and Yongfeng Zhang. Autoflow: Automated workflow generation for large language model agents. *arXiv preprint arXiv:2407.12821*, 2024.
- Fei Liu, Tong Xialiang, Mingxuan Yuan, Xi Lin, Fu Luo, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. Evolution of heuristics: Towards efficient automatic algorithm design using large language model. In *Forty-first International Conference on Machine Learning*, 2024.
- Zijun Liu, Yanzhe Zhang, Peng Li, Yang Liu, and Diyi Yang. Dynamic llm-agent network: An llm-agent collaboration framework with agent team optimization. *arXiv preprint arXiv:2310.02170*, 2023.
- Chris Lu, Sebastian Towers, and Jakob Foerster. Arbitrary order meta-learning with simple population-based evolution. In *ALIFE 2023: Ghost in the Machine: Proceedings of the 2023 Artificial Life Conference*. MIT Press, 2023.

- Chris Lu, Samuel Holt, Claudio Fanconi, Alex J Chan, Jakob Foerster, Mihaela van der Schaar, and Robert Tjarko Lange. Discovering preference optimization algorithms with and for large language models. *arXiv preprint arXiv:2406.08414*, 2024a.
- Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024b.
- Cong Lu, Shengran Hu, and Jeff Clune. Intelligent go-explore: Standing on the shoulders of giant foundation models. *arXiv preprint arXiv:2405.15143*, 2024c.
- Zhichao Lu, Ian Whalen, Vishnu Boddeti, Yashesh Dhebar, Kalyanmoy Deb, Erik Goodman, and Wolfgang Banzhaf. Nsga-net: neural architecture search using multi-objective genetic algorithm. In *Proceedings of the genetic and evolutionary computation conference*, pp. 419–427, 2019.
- Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. In *The Twelfth International Conference on Learning Representations*, 2023.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36, 2024.
- Elliot Meyerson, Mark J Nelson, Herbie Bradley, Adam Gaier, Arash Moradi, Amy K Hoover, and Joel Lehman. Language model crossover: Variation through few-shot prompting. *arXiv preprint arXiv:2302.12170*, 2023.
- Shen-yun Miao, Chao-Chun Liang, and Keh-Yih Su. A diverse corpus for evaluating and developing english math word problem solvers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 975–984, 2020.
- Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015.
- Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. Webgpt: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2021.
- Andrew Ng. Issue 253. <https://www.deeplearning.ai/the-batch/issue-253/>, June 2024. Newsletter issue.
- Ben Norman and Jeff Clune. First-explore, then exploit: Meta-learning intelligent exploration. *arXiv preprint arXiv:2307.02276*, 2023.
- OpenAI. Introducing chatgpt. <https://openai.com/index/chatgpt/>, November 2022. Blog post.
- OpenAI. Simple evals, 2023. URL <https://github.com/openai/simple-evals>. Accessed: 2024-08-10.
- OpenAI. Gpt-4 technical report, 2024.
- Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pp. 1–22, 2023.
- Arkil Patel, Satwik Bhattamishra, and Navin Goyal. Are NLP models really able to solve simple math word problems? In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 2080–2094, Online, June 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.naacl-main.168.

- Chen Qian, Xin Cong, Cheng Yang, Weize Chen, Yusheng Su, Juyuan Xu, Zhiyuan Liu, and Maosong Sun. Communicative agents for software development. *arXiv preprint arXiv:2307.07924*, 2023.
- Chen Qian, Zihao Xie, Yifei Wang, Wei Liu, Yufan Dang, Zhuoyun Du, Weize Chen, Cheng Yang, Zhiyuan Liu, and Maosong Sun. Scaling large-language-model-based multi-agent collaboration. *arXiv preprint arXiv:2406.07155*, 2024.
- Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-Rong Wen. Tool learning with large language models: A survey. *arXiv preprint arXiv:2405.17935*, 2024.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36, 2024.
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. Gpqa: A graduate-level google-proof q&a benchmark, 2023.
- Toran Bruce Richards. Autogpt. <https://github.com/Significant-Gravitas/AutoGPT>, 2023. GitHub repository.
- Tim Rocktäschel. *Artificial Intelligence: 10 Things You Should Know*. Seven Dials, September 2024. ISBN 978-1399626521.
- Md Omar Faruk Rokon, Risul Islam, Ahmad Darki, Evangelos E Papalexakis, and Michalis Faloutsos. SourceFinder: Finding malware Source-Code from publicly available repositories in GitHub. In *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020)*, pp. 149–163, 2020.
- Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=Yacmpz84TH>.
- Jürgen Schmidhuber. Evolutionary principles in self-referential learning. on learning now to learn: The meta-meta-meta...-hook. Diploma thesis, Technische Universität München, Germany, 14 May 1987. URL <http://www.idsia.ch/~juergen/diploma.html>.
- Jürgen Schmidhuber. Gödel machines: self-referential universal problem solvers making provably optimal self-improvements. *arXiv preprint cs/0309048*, 2003.
- Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, Aayush Gupta, HyoJung Han, Sevien Schulhoff, et al. The prompt report: A systematic survey of prompting techniques. *arXiv preprint arXiv:2406.06608*, 2024.
- Xuan Shen, Yaohua Wang, Ming Lin, Yilun Huang, Hao Tang, Xiuyu Sun, and Yanzhi Wang. Deepmad: Mathematical architecture design for deep convolutional neural network. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 6163–6173, 2023.
- Freda Shi, Mirac Suzgun, Markus Freitag, Xuezhi Wang, Suraj Srivats, Soroush Vosoughi, Hyung Won Chung, Yi Tay, Sebastian Ruder, Denny Zhou, Dipanjan Das, and Jason Wei. Language models are multilingual chain-of-thought reasoners. In *The Eleventh International Conference on Learning Representations*, 2023.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023.

- Kenneth O Stanley and Joel Lehman. *Why greatness cannot be planned: The myth of the objective*. Springer, 2015.
- Kenneth O Stanley, Jeff Clune, Joel Lehman, and Risto Miikkulainen. Designing neural networks through neuroevolution. *Nature Machine Intelligence*, 1(1):24–35, 2019.
- Richard S. Sutton. The bitter lesson, 2019. URL <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>.
- Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.
- Jon Turow. The rise of ai agent infrastructure. June 2024. URL <https://www.madrona.com/the-rise-of-ai-agent-infrastructure/>.
- Sai Vemprala, Rogerio Bonatti, Arthur Buckner, and Ashish Kapoor. Chatgpt for robotics: Design principles and model abilities. Technical Report MSR-TR-2023-8, Microsoft, February 2023. URL <https://www.microsoft.com/en-us/research/publication/chatgpt-for-robotics-design-principles-and-model-abilities/>.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv: Arxiv-2305.16291*, 2023a.
- Jane X Wang, Zeb Kurth-Nelson, Dhruva Tirumala, Hubert Soyer, Joel Z Leibo, Remi Munos, Charles Blundell, Dhharshan Kumaran, and Matt Botvinick. Learning to reinforcement learn. *arXiv preprint arXiv:1611.05763*, 2016.
- Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.
- Rui Wang, Joel Lehman, Jeff Clune, and Kenneth O. Stanley. Poet: open-ended coevolution of environments and their optimized solutions. In *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO ’19*, pp. 142–151, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450361118. doi: 10.1145/3321707.3321799.
- Rui Wang, Joel Lehman, Aditya Rawal, Jiale Zhi, Yulun Li, Jeffrey Clune, and Kenneth Stanley. Enhanced poet: Open-ended reinforcement learning through unbounded invention of learning challenges and their solutions. In *International conference on machine learning*, pp. 9940–9951. PMLR, 2020.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*, 2023b.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation framework. *arXiv preprint arXiv:2308.08155*, 2023.
- Benfeng Xu, An Yang, Junyang Lin, Quan Wang, Chang Zhou, Yongdong Zhang, and Zhendong Mao. Expertprompting: Instructing large language models to be distinguished experts. *arXiv preprint arXiv:2305.14688*, 2023.
- Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=Bb4VGOWELI>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

- Wenhao Yu, Nimrod Gileadi, Chuyuan Fu, Sean Kirmani, Kuang-Huei Lee, Montserrat Gonzalez Arenas, Hao-Tien Lewis Chiang, Tom Erez, Leonard Hasenclever, Jan Humplik, et al. Language to rewards for robotic skill synthesis. In *Conference on Robot Learning*, pp. 374–404. PMLR, 2023.
- Siyu Yuan, Kaitao Song, Jiangjie Chen, Xu Tan, Dongsheng Li, and Deqing Yang. Evoagent: Towards automatic multi-agent generation via evolutionary algorithms. *arXiv preprint arXiv:2406.14228*, 2024.
- Eliezer Yudkowsky et al. Artificial Intelligence as a positive and negative factor in global risk. *Global catastrophic risks*, 1(303):184, 2008.
- Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. Textgrad: Automatic” differentiation” via text. *arXiv preprint arXiv:2406.07496*, 2024.
- Matei Zaharia, Omar Khattab, Lingjiao Chen, Jared Quincy Davis, Heather Miller, Chris Potts, James Zou, Michael Carbin, Jonathan Frankle, Naveen Rao, and Ali Ghodsi. The shift from models to compound ai systems. <https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/>, 2024.
- Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah Goodman. STar: Bootstrapping reasoning with reasoning. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho (eds.), *Advances in Neural Information Processing Systems*, 2022. URL https://openreview.net/forum?id=_3ELRdg2sgI.
- Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (stop): Recursively self-improving code generation. In *First Conference on Language Modeling*, 2024.
- Jenny Zhang, Joel Lehman, Kenneth Stanley, and Jeff Clune. OMNI: Open-endedness via models of human notions of interestingness. In *The Twelfth International Conference on Learning Representations*, 2024a. URL <https://openreview.net/forum?id=AgM3MzT99c>.
- Shaokun Zhang, Jieyu Zhang, Jiale Liu, Linxin Song, Chi Wang, Ranjay Krishna, and Qingyun Wu. Offline training of language model agents with functions as learnable weights. In *Forty-first International Conference on Machine Learning*, 2024b.
- Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model based agents. *arXiv preprint arXiv:2404.13501*, 2024c.
- Huaixiu Steven Zheng, Swaroop Mishra, Xinyun Chen, Heng-Tze Cheng, Ed H Chi, Quoc V Le, and Denny Zhou. Take a step back: Evoking reasoning via abstraction in large language models. *arXiv preprint arXiv:2310.06117*, 2023.
- Pei Zhou, Jay Pujara, Xiang Ren, Xinyun Chen, Heng-Tze Cheng, Quoc V Le, Ed H Chi, Denny Zhou, Swaroop Mishra, and Huaixiu Steven Zheng. Self-discover: Large language models self-compose reasoning structures. *arXiv preprint arXiv:2402.03620*, 2024a.
- Wangchunshu Zhou, Yixin Ou, Shengwei Ding, Long Li, Jialong Wu, Tiannan Wang, Jiamin Chen, Shuai Wang, Xiaohua Xu, Ningyu Zhang, et al. Symbolic learning enables self-evolving agents. *arXiv preprint arXiv:2406.18532*, 2024b.
- Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. Gptswarm: Language agents as optimizable graphs. In *Forty-first International Conference on Machine Learning*, 2024.
- Luisa Zintgraf, Sebastian Schulze, Cong Lu, Leo Feng, Maximilian Igl, Kyriacos Shiarlis, Yarin Gal, Katja Hofmann, and Shimon Whiteson. Varibad: Variational bayes-adaptive deep rl via meta-learning. *Journal of Machine Learning Research*, 22(289):1–39, 2021a.
- Luisa M Zintgraf, Leo Feng, Cong Lu, Maximilian Igl, Kristian Hartikainen, Katja Hofmann, and Shimon Whiteson. Exploration in approximate hyper-state space for meta reinforcement learning. In *International Conference on Machine Learning*, pp. 12991–13001. PMLR, 2021b.

补充材料

目录

泛化与可迁移性 20

B 提示词 21

C 框架代码 23

ARC 挑战 25 的实验细节

E 推理与问题求解领域的实验细节 29

F 基线 30

G 示例智能体 31

H 元代理搜索的伪代码 33

I 初始化的影响 33

J 实验成本 34

A 泛化与可迁移性

本节中，我们更详细地介绍第 4.3 节中的实验，并给出跨不同领域迁移智能体的完整结果。

对于表 3 所示结果，我们使用“gpt-4o-2024-05-13”作为 GPT-4，“claude-3-haiku-20240307”作为 Claude-Haiku，“claude-3-5-sonnet-20240620”作为 Claude-Sonnet。

表 4：将 MGSM 中的顶级智能体迁移到其他数学领域时，在不同数学领域上的性能。元智能体搜索发现的智能体在不同数学领域上始终优于基线。我们报告测试准确率和 95% 自助置信区间。顶级智能体的名称由元智能体搜索生成。

Agent Name	Accuracy (%)				
	MGSM	GSM8K	GSM-Hard	SVAMP	ASDiv
Manually Designed Agents					
Chain-of-Thought (Wei et al., 2022)	28.0 ± 3.1	34.9 ± 3.2	15.0 ± 2.5	77.8 ± 2.8	88.9 ± 2.2
COT-SC (Wang et al., 2023b)	28.2 ± 3.1	37.8 ± 3.4	15.5 ± 2.5	78.2 ± 2.8	89.0 ± 2.1
Self-Refine (Madaan et al., 2024)	27.5 ± 3.1	38.9 ± 3.4	15.1 ± 2.4	78.5 ± 2.8	89.2 ± 2.2
LLM Debate (Du et al., 2023)	39.0 ± 3.4	43.6 ± 3.4	17.4 ± 2.6	76.0 ± 3.0	88.9 ± 2.2
Step-back Abstraction (Zheng et al., 2023)	31.1 ± 3.2	31.5 ± 3.3	12.2 ± 2.3	76.1 ± 3.0	87.8 ± 2.3
Quality-Diversity (Lu et al., 2024c)	23.8 ± 3.0	28.0 ± 3.1	14.1 ± 2.4	69.8 ± 3.2	80.1 ± 2.8
Role Assignment (Xu et al., 2023)	30.1 ± 3.2	37.0 ± 3.4	18.0 ± 2.7	73.0 ± 3.0	83.1 ± 2.6
Top Agents Searched on MGSM (Math)			Transferred within Math Domains		
Dynamic Role-Playing Architecture	53.4 ± 3.5	69.5 ± 3.2	31.2 ± 3.2	81.5 ± 2.6	91.8 ± 1.8
Structured Multimodal Feedback Loop	50.2 ± 3.5	64.5 ± 3.4	30.1 ± 3.2	82.6 ± 2.6	89.9 ± 2.1
Interactive Multimodal Feedback Loop	47.4 ± 3.5	64.9 ± 3.3	27.6 ± 3.2	80.6 ± 2.8	89.8 ± 2.1

表 5：将数学（MGSM）领域的顶级智能体迁移到非数学领域时，在多个领域上的性能。元智能体搜索在数学领域发现的智能体迁移到数学之外的领域后，可以优于或匹配基线的性能。我们报告测试准确率和 95% 自助置信区间。

Agent Name	Accuracy (%)	F1 Score		Accuracy (%)	
	Math	Reading Comprehension	Multi-task	Science	
Manually Designed Agents					
Chain-of-Thought (Wei et al., 2022)	28.0 ± 3.1	64.2 ± 0.9	65.4 ± 3.3	29.2 ± 3.1	
COT-SC (Wang et al., 2023b)	28.2 ± 3.1	64.4 ± 0.8	65.9 ± 3.2	30.5 ± 3.2	
Self-Refine (Madaan et al., 2024)	27.5 ± 3.1	59.2 ± 0.9	63.5 ± 3.4	31.6 ± 3.2	
LLM Debate (Du et al., 2023)	39.0 ± 3.4	60.6 ± 0.9	65.6 ± 3.3	31.4 ± 3.2	
Step-back Abstraction (Zheng et al., 2023)	31.1 ± 3.2	60.4 ± 1.0	65.1 ± 3.3	26.9 ± 3.0	
Quality-Diversity (Lu et al., 2024c)	23.8 ± 3.0	61.8 ± 0.9	65.1 ± 3.1	30.2 ± 3.1	
Role Assignment (Xu et al., 2023)	30.1 ± 3.2	65.8 ± 0.9	64.5 ± 3.3	31.1 ± 3.1	
Top Agents Searched on Math (MGSM)		Transferred beyond Math Domains			
Dynamic Role-Playing Architecture	53.4 ± 3.5	70.4 ± 0.9	62.4 ± 3.4	28.6 ± 3.1	
Structured Multimodal Feedback Loop	50.2 ± 3.5	70.4 ± 0.9	67.0 ± 3.2	28.7 ± 3.1	
Interactive Multimodal Feedback Loop	47.4 ± 3.5	71.9 ± 0.8	64.8 ± 3.3	29.9 ± 3.2	

我们将从 MGSM（数学）领域发现的智能体迁移到其他数学领域，以测试所发明的智能体能否跨不同领域泛化。类似地，我们测试 MGSM 中的前 3 个智能体，并将它们迁移到（1）四个流行的数学领域：GSM8K（Cobbe 等人，2021）、GSM-Hard（Gao 等人，2023）、SVAMP（Patel 等人，2021）和 ASDiv（Miao 等人，2020），以及（2）第 4.2 节采用的三个数学之外的领域。如表 4 所示，我们观察到元智能体搜索相比基线具有类似的性能优势。更令人惊讶的是，我们观察到在数学领域发现的智能体可以迁移到非数学领域（表 5）。虽然最初在数学领域搜索的智能体的性能不能完全匹配专门为目标领域设计的智能体，但它们仍然优于（在阅读理解和多任务中）或匹配（在科学中）最先进的手工设计智能体基线。这些结果表明，元智能体搜索可以发现可泛化的设计模式和智能体系统。

B 提示词

我们在元代理搜索中为元代理使用以下提示词。根据领域和迭代而变化的提示词中的变量已突出显示。

We use the following system prompt for every query in the meta agent.

System prompt for the meta agent.

You are a helpful assistant. Make sure to return in a WELL-FORMED JSON object.

我们使用以下提示词，让元代理基于先前发现的代理档案来设计新代理。

Main prompt for the meta agent.

您是一位专业的机器学习研究者，正在测试各种代理系统。您的目标是在这些系统内设计诸如提示词和工作流之类的构建模块，以解决复杂任务。您的目标是设计一个在 [领域简要描述] 上表现良好的最优代理。

[Framework Code]

[Output Instructions and Examples]

[Discovered Agent Archive] (initialized with baselines, updated at every iteration)

您的任务 您对文献中的提示技术和代理工作有深入了解。您的目标是通过提出有趣的新代理来最大化指定的性能指标。仔细观察已发现的代理，思考可以从中获得哪些洞见、经验或垫脚石。在思考下一个值得尝试的有趣代理时，请发挥创造力。鼓励您从相关代理论文或其他研究领域的学术论文中汲取灵感。利用档案中的知识和学术文献的启发，提出下一个有趣的代理系统设计。跳出固有思维模式。

领域描述见附录 D 和 E，框架代码见附录 C。我们使用以下提示词来指导元代理的输出并规范其格式。在此，我们收集并展示元代理在提示词中可能犯的一些常见错误。我们发现这能有效提高生成代码的质量。这些格式提示词受 Lu 等人（2024a）启发。

Output Instruction and Example.

输出说明与示例：第一个键应为（“thought”），它应捕捉您设计下一个函数的思考过程。在“thought”部分，首先推理下一个值得尝试的有趣代理应该是什么，然后描述您的推理和代理设计背后的整体概念，最后详述实现步骤。第二个键（“name”）对应您下一个代理架构的名称。最后，最后一个键（“code”）对应您想尝试的 Python 代码中确切的“forward()”函数。您必须在“code”中编写完整代码：您的代码将成为整个项目的一部分，因此请实现完整、可靠、可复用的代码片段。

以下是下一个代理的输出格式示例：{“思考”：“**洞见：** 你对下一个有趣智能体的洞见。”**总体思路：** 你的推理以及智能体设计背后的整体概念。”**实现：** 逐步描述实现过程。”，“名称”：“你提出的智能体名称”，“代码”：“def forward(self, taskInfo): # 在此处编写代码”}}

WRONG Implementation examples:

[Examples of potential mistakes the meta agent may make in implementation]

在元智能体首次响应之后，我们进行两轮自我反思，以使生成的智能体新颖且无错误（Shinn 等人，2023；Madaan 等人，2024）。

Prompt for self-reflection round 1.

[来自上一轮迭代的生成智能体] 仔细审查所提出的新架构，并反思以下几点：

1. ****趣味性****：评估你所提出的架构与档案中现有方法相比是否有趣或具有创新性。如果你确定所提出的架构不够有趣，请提出一个能够解决这些不足的新架构。
 - 务必检查所提出的架构与先前尝试之间的差异。
 - 仔细比较该提案与档案中的架构，包括它们在实现上的实际差异。
 - 判断当前架构是否具有创新性。
 - 运用批判性思维！
2. ****实现错误****：识别您在实现过程中可能犯的任何错误。仔细审查代码，调试您发现的任何问题，并提供修正版本。请务必检查提示中的“## 错误实现示例”。
3. ****改进****：基于所提出的架构，在详细实现中提出可提升其性能或有效性的改进建议。在此步骤中，专注于在不改变整体设计框架的前提下优化和完善现有实现，除非您想提出一个不同的架构（如果当前架构不够有趣）。
 - 仔细观察实现是否真正完成了其应做的工作。
 - 检查实现中是否存在冗余代码或不必要的步骤。用有效的实现替换它们。
 - 尽量避免实现与之前的智能体过于相似。

然后，您需要基于反思来改进或修订实现，或者实现新提出的架构。

您的回答应按如下方式组织：

“反思”：提供您对架构有趣性的思考，识别实现中的任何错误，并提出改进建议。“思考”：如有必要，修订您之前的提案或提出新架构，使用与示例回答相同的格式。“名称”：为修订后的或新架构提供一个名称。（不要在名称中使用“新”或“改进”等词。）“代码”：提供修正后的代码或改进后的实现。确保您在此代码中实际实现了您的修正和改进。

Prompt for self-reflection round 2.

利用“## 错误实现示例”部分的提示，进一步修改代码。您的回复应按如下方式组织：在“反思”中包含更新后的反思。重复之前的“思考”和“名称”。在“代码”部分更新修正后的代码版本。

当生成的代码执行过程中遇到错误时，我们进行反思并重新运行代码。如果错误持续存在，此过程最多重复五次。以下是我们用于自我反思任何运行时错误的提示：

Prompt for self-reflection when a runtime error occurs.

评估期间出错: [运行时错误] 仔细考虑您在最新实现中哪里出错了。利用之前尝试的洞见, 尝试调试当前代码以实现相同的思路。在“思考”中重复您之前的思考, 并将调试思路放入“调试思考”中。

C 框架代码

在本文中, 我们为元代理提供了一个简单的框架来实现基本功能, 例如查询基础模型 (Foundation Models, FMs) 和格式化提示。该框架包含不到 100 行代码 (不包括注释)。在此框架中, 我们将每条信息封装到一个命名元组 Info 对象中, 从而便于组合不同类型的信息 (例如, FM 响应、工具函数调用的结果、任务描述), 并促进不同模块之间的通信。此外, 在 FM 模块中, 我们通过将所有输入的 Info 对象连接成结构化格式来自动构建提示, 每个 Info 以其元数据 (例如, 名称、作者) 作为标题。在整个附录中, 我们重命名了代码中的一些变量, 以匹配正文中使用的术语。

代码 1: 元智能体搜索中使用的简单框架。# 用于保存任务信息的命名元组 Info = 命名元组 ('信息', ['名称', '作者', '内容', '迭代_idx'])

```
# FM 响应的格式说明 FORMAT_INST = lambda request_keys: f'Reply EXACTLY with the following JSON
format.\n{str(request_keys)}\nDO NOT MISS ANY FIELDS AND MAKE SURE THE JSON FORMAT IS
CORRECT!\n"
```

```
# 基础模型模块角色描述 ROLE_DESC = lambda role: f'你是一名
{role}。"
```

```
@backoff.on_exception(backoff.expo, openai.RateLimitError) def get_json_response_from_gpt(msg, model,
system_message, temperature): \ """ 从 GPT 模型获取 JSON 响应的函数。
```

参数:

- msg (str): 用户消息。
- model (str): 要使用的模型。
- system_message (str): 系统消息。
- temperature (float): 采样温度。

返回:

- dict: JSON 响应。

```
\ """
```

```
...
```

```
返回 json_dict
```

类 FM_Module: \ """ 基础类, 用于功能模块 (FM module) 。

属性:

- output_fields (列表): 输出中预期的字段。
- 名称 (字符串): 功能模块的名称。
- 角色 (字符串): 功能模块的角色描述。
- 模型 (字符串): 要使用的模型。
- temperature (float): 采样温度。
- 标识符 (字符串): 功能模块实例的唯一标识符。

```
\ """
```

定义 `__init__(self, output_fields: 列表, 名称: 字符串, role='helpful 助手', model='gpt-3.5-turbo-0125', temperature=0.5) -> 无: ...`

`def generate_prompt(self, input_infos, 指令) -> 字符串: \ """ 为 FM 生成提示。`

参数:

- `input_infos` (列表): 输入信息列表。

- 指令 (字符串): 任务指令。

返回:

- 元组: 系统提示和用户提示。

生成提示的示例: "" 你是一个乐于助人的助手。

输出格式: 请严格按以下 JSON 格式回复。...

你的任务: 你将获得若干成对的示例输入和输出。输出...

思考 #1 由思维链 `hkFo` (你自己): ...

指令: 请逐步思考, 然后通过编写代码来解决问题。

""
\ ""

...

返回 `system_prompt`, 提示

`def query(self, input_infos: list, instruction, iteration_idx=-1) -> list[信息]: \ """ 使用提供的输入信息和指令查询基础模型。`

参数:

- `input_infos` (列表): 输入信息列表。

- 指令 (字符串): 任务指令。

- `iteration_idx` (int): 任务的迭代索引。

返回: - `output_infos` (列表 [信息]): 输出信息。

\ ""

...

返回 `output_infos`

定义 `__repr__(self): 返回 f'{self.agent_name} {自身标识}'`

定义 `__call__(self, input_infos: 列表, 指令, iteration_idx=-1): 返回 self.query(input_infos, 指令, iteration_idx=iteration_idx)`

类 智能体系统: 定义 前向传播 (自身, 任务信息) -> 联合类型 [信息, 字符串]: \ ""
处理任务信息的占位方法。

```

参数：
- taskInfo (Info)：任务信息。

返回：
- 答案（联合类型 [信息, 字符串]）：您的最终答案。返回任一
  命名元组 Info 或字符串作为答案。
\ """
pass

```

借助所提供的框架，可以轻松地通过“forward”函数定义智能体。此处展示一个利用该框架实现自我反思的示例。

代码 2：自我反思实现示例

```

def forward(self, taskInfo): # 初始推理指令 cot_initial_instruction = "请逐步思考，然后解决任务。"

    # 关于反思先前尝试与反馈以改进的指令 cot_reflect_instruction = "鉴于先前的尝试与反馈，请仔细思考
    # 您最近一次尝试中可能出错之处。利用先前尝试的洞见，尝试更好地解决任务。" cot_module =
    FM_Module(['thinking', '答案'], '思维链')

    # 关于提供反馈并纠正答案的指令 critic_instruction = "请审阅上述答案并批评其中可能存在的错误。如果
    # 您绝对确信其正确，请在'正确'中输出'真'。" critic_module = FM_Module(['feedback', '正确'], '批评者')

    N_max = 5 # Maximum number of attempts

    # 初始尝试 cot_inputs = [任务信息] 思考, 答案 = cot_module(cot_inputs, cot_initial_instruction, 0)

    对于 i 在范围内 (N_max):
        # 从批评者处获取反馈与正确状态 反馈, 正确 = critic_module([taskInfo, 思考, 答案],
        critic_instruction, i) 如果 正确.内容 == '真': 中断

        # 将反馈添加到下一次迭代的输入中 cot_inputs.extend([thinking, 答案, 反馈])

        # 反思先前的尝试并完善答案 思考, 答案 = cot_module(cot_inputs, cot_reflect_instruction
        , i + 1)

    返回 答案

```

D ARC 挑战实验细节

ARC 挑战中的一个示例任务如图 4 所示。在 ARC 挑战实验（第 4.1 节）中，我们将网格表示为二维数组的字符串，其中每种颜色由一个整数表示。我们指示元代理设计生成代码作为解决方案的代理，而不是直接输出答案。此外，我们在框架内提供了两个工具函数：

- (1) 测试生成的代码能否解决示例网格，以及
- (2) 获取任务的

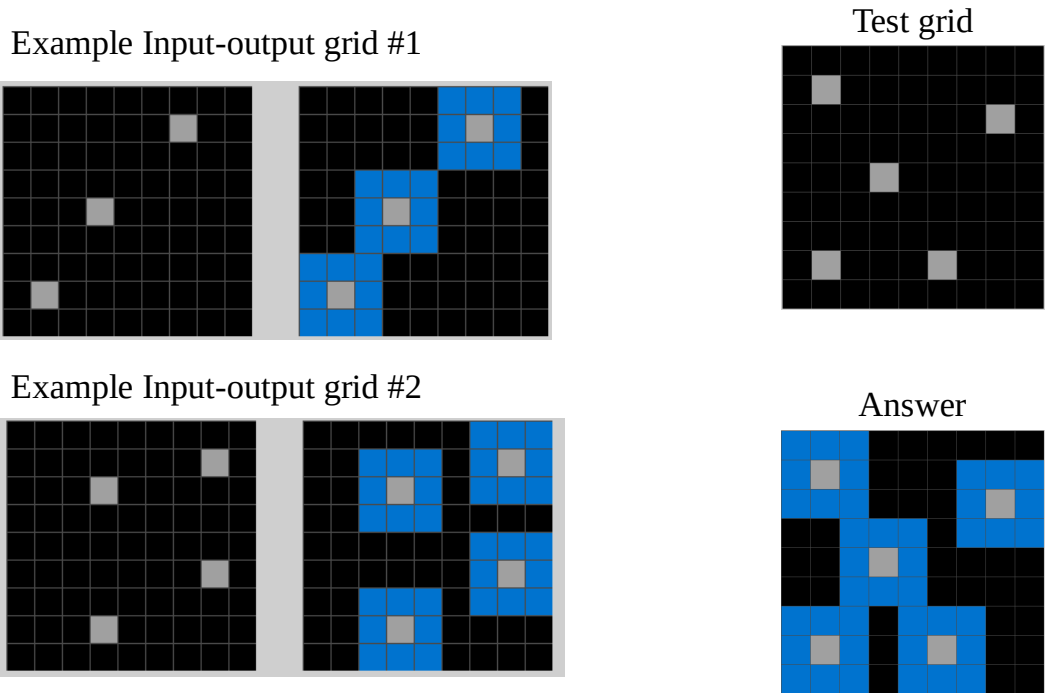


图 4：ARC 挑战中的一个示例任务（Chollet, 2019）。给定输入-输出网格示例，AI 系统被要求学习变换规则，然后将这些学到的规则应用于测试网格以预测最终答案。

答案，通过将生成的代码应用于测试网格。准确率通过参考解与预测答案之间的精确匹配来计算。元代理使用“gpt-4o-2024-05-13”（OpenAI, 2024），而发现的代理和基线使用“gpt-3.5-turbo-0125”（OpenAI, 2022）进行评估，以降低计算成本。

ARC 的领域描述如下所示：

Description of ARC for the meta agent.

你的目标是找到一个在 ARC（抽象与推理语料库）挑战中表现良好的最优代理。在此挑战中，每个任务包含三个演示示例和一个测试示例。每个示例由一个“输入网格”和一个“输出网格”组成。测试者需要使用从示例中学到的变换规则来预测测试示例的输出网格。

An example task from ARC challenge:

任务概述：你将获得若干配对的示例输入和输出网格。输出是通过对输入网格应用变换规则产生的。除了配对的示例输入和输出之外，还有一个没有已知输出的测试输入。输入和输出都是“网格”。网格是一个由 0 到 9（含）之间的整数组成的矩形矩阵。每个数字对应一种颜色。0 是黑色。你的任务是从示例中确定变换规则并找出答案，包括确定测试输出网格的大小，并用适当的颜色或数字正确填充网格的每个单元格。

变换只需明确且适用于示例输入和测试输入，无需对所有可能输入都有效。仔细观察示例，在脑海中想象网格，并尝试找出规律。

```

## Examples:
### Example 0:
input = [[0,0,0,0,5,0,0,0,0], [0,0,0,0,5,0,0,0,0], [0,0,0,4,5,0,0,0,0], [0,0,0,4,5,4,4,0,0], [0,0,3,3,5,0,0,0,0],
[0,0,0,3,5,0,0,0,0], [0,0,0,3,5,3,3,3,0], [0,0,0,3,5,0,0,0,0], [0,0,0,0,5,0,0,0,0], [0,0,0,0,5,0,0,0,0]]
output = [[0,0,0,0], [0,0,0,0], [0,0,0,4], [0,0,4,4], [0,0,3,3], [0,0,0,3], [0,3,3,3], [0,0,0,3], [0,0,0,0],
[0,0,0,0]]

### Example 1:
input = [[0,0,0,0,5,0,0,0,0], [0,0,0,2,5,0,0,0,0], [0,0,0,2,5,2,6,0,0], [0,0,0,2,5,0,0,0,0], [0,0,0,2,5,2,2,2,0],
[0,0,6,6,5,6,0,0,0], [0,0,0,2,5,0,0,0,0], [0,2,2,0,5,2,0,0,0], [0,0,0,2,5,0,0,0,0], [0,0,0,0,5,0,0,0,0]]
output = [[0,0,0,0], [0,0,0,2], [0,0,6,2], [0,0,0,2], [0,2,2,2], [0,0,6,6], [0,0,0,2], [0,2,2,2], [0,0,0,2],
[0,0,0,0]]

### Example 2:
input = [[0,0,0,0,5,0,0,0,0], [0,0,0,0,5,7,0,0,0], [0,0,0,8,5,0,0,0,0], [0,0,0,8,5,0,0,0,0], [0,7,8,8,5,0,0,0,0],
[0,0,0,0,5,8,8,0,0], [0,0,0,8,5,0,0,0,0], [0,0,0,8,5,0,0,0,0], [0,0,0,0,5,8,7,0,0], [0,0,0,0,5,0,0,0,0]]
output= [[0,0,0,0], [0,0,0,7], [0,0,0,8], [0,0,0,8], [0,7,8,8], [0,0,8,8], [0,0,0,8], [0,0,0,8], [0,0,7,8],
[0,0,0,0]]

### Test Problem:
input = [[0,0,0,0,5,0,0,0,0], [0,0,0,1,5,0,0,0,0], [0,0,0,1,5,1,0,0,0], [0,1,1,1,5,1,1,1,6], [0,0,0,6,5,6,6,0,0],
[0,0,0,0,5,1,1,1,0], [0,0,0,1,5,0,0,0,0], [0,0,0,1,5,1,6,0,0], [0,0,0,0,5,6,0,0,0], [0,0,0,0,5,0,0,0,0]]

```

根据提供的示例分析变换规则，并确定测试问题的输出结果。

Here we present the best agent on ARC discovered by Meta Agent Search.

代码 3: 元智能体搜索在抽象推理语料库上发现的最佳智能体 # 结构化反馈与集成智能体 定义 前向传播 (自身, 任务信息): # 步骤 1: 使用多个基础模型模块生成初始候选解 initial_instruction = '请逐步思考, 然后通过编写代码来解决问题.' num_candidates = 5 # 初始候选数量 initial_module = [FM_Module(['thinking', '代码'], '初始解', temperature=0.8) 对于 _ 在 range(num_candidates)] 中

```

initial_solutions = [] 对于 i 在 range(num_candidates): 思考 = initial_module[i](任务信息,
initial_instruction) 思考, 代码 = 思考 [0], 思考 [1] 反馈, correct_examples, wrong_examples = 自身。
run_examples_and_get_feedback(code) 如果 len(correct_examples) > 0: # 仅考虑至少通过一个示例的解
initial_solutions.append({'thinking': 思考, '代码': 代码, '反馈': 反馈, 'correct_count': 长度
(correct_examples)}))

```

```

# 步骤 2: 为每个候选解模拟人类反馈 human_like_feedback_module = FM_Module(['thinking',
'feedback'], '人类式反馈', temperature=0.5) human_feedback_instruction = '请为代码提供人类式反馈, 重
点关注常见错误、启发式修正和最佳实践.'

```

```

对于 sol 在 initial_solutions: 思考 = human_like_feedback_module([taskInfo, sol['思考'], sol['代码']],
human_feedback_instruction) human_thinking, human_feedback = 思考 [0], 思考 [1] sol["人类反馈"] =
human_feedback

```

```
# 步骤 3: 分配专家顾问进行评估并提供针对性反馈 expert_roles = ['效率专家', '可读性专家', '简洁性专家'] expert_advisors = [FM_Module(['thinking', '反馈'], 角色, temperature=0.6) 对于角色在 expert_roles] expert_instruction = '请评估给定代码并提供改进的针对性反馈。'
```

```
对于 sol 在 initial_solutions: sol_feedback = {} 对于顾问在 expert_advisors: 思考 = 顾问 ([任务信息, sol['思考'], sol['代码']], expert_instruction) 思考, 反馈 = 思考 [0], 思考 [1] sol_feedback[顾问角色] = 反馈 sol["专家反馈"] = sol_feedback
```

```
# 步骤 4: 解析并结构化反馈以避免冗余, 并迭代优化解决方案 max_refinement_iterations = 3 refinement_module = FM_Module(['thinking', '代码'], '优化模块', temperature=0.5) refined_solutions = []
```

```
对于 sol 在 initial_solutions: 对于 i 在 range(max_refinement_iterations): combined_feedback = sol['反馈'].内容 + sol["人类反馈"].content + ".join([fb.内容 对于 fb 在 sol["专家反馈"].values()]) structured_feedback = ".join(set(combined_feedback.split())) # 避免冗余 refinement_instruction = '使用结构化反馈, 优化解决方案以提升其性能.' 思考 = refinement_module([taskInfo, sol['思考'], sol['代码'], 信息 ('反馈', '结构化反馈', structured_feedback, i)], refinement_instruction, i) refinement_thinking, refined_code = 思考 [0], 思考 [1] 反馈, correct_examples, wrong_examples = 自身.run_examples_and_get_feedback(refined_code) 如果 len(correct_examples) > 0: sol.更新 ({'思考': refinement_thinking, '代码': refined_code, '反馈': 反馈, 'correct_count': len(correct_examples)}) refined_solutions.append(sol)
```

```
# 步骤 5: 选择性能最佳的解决方案, 并使用集成方法做出最终决策 sorted_solutions = sorted(refined_solutions, key=lambda x : x['正确计数'], reverse=True) top_solutions = sorted_solutions[:3] # 选择前 3 个解决方案
```

```
final_decision_instruction = '鉴于上述所有解决方案, 请仔细推理并给出最终答案, 通过编写代码实现.' final_decision_module = refinement_module(['thinking', '代码'], '最终决策模块', temperature=0.1) final_inputs = [任务信息] + [针对 top_solutions 中每个解决方案的条目, 包括解决方案的'思考'、'代码'、'反馈'] final_thoughts = final_decision_module(final_inputs, final_decision_instruction) final_thinking, final_code = 最终思考 [0], 最终思考 [1] 答案 = self.get_test_output_from_code(final_code)
```

推理与问题求解的实验细节

领域

为了降低搜索和评估过程中的成本，我们从每个领域抽取数据子集。对于 GPQA（科学），我们使用 GPQA 钻石集，验证集包含 32 个问题，其余 166 个问题构成测试集。对于其他领域，验证集和测试集分别抽取 128 个和 800 个问题。我们对 GPQA 评估智能体五次，对其他领域评估一次，以保持评估总次数一致。每个领域使用零样本风格的问题，除了 DROP（阅读理解）遵循（OpenAI，2023）的做法使用单样本风格的问题。元智能体使用“gpt-4o-2024-05-13”（OpenAI，2024），而发现的智能体和基线使用“gpt-3.5-turbo-0125”（OpenAI，2022）进行评估，以降低计算成本。

We present the description of each domain we provide to the meta agent.

Description of DROP (Reading Comprehension).

您的目标是找到一个在段落离散推理阅读理解基准（DROP）上表现优异的智能体，该基准评估跨多个段落进行离散推理和理解详细信息的能力。

An example question from DROP:

You will be asked to read a passage and answer a question.

段落：非本国国民占巴林人口的一半以上，移民约占总人口的 55%。其中，绝大多数来自南亚和东南亚：根据 2005 年至 2009 年间的各种媒体报道和政府统计数据，大约有 29 万印度人、12.5 万孟加拉国人、4.5 万巴基斯坦人、4.5 万菲律宾人和 8000 印度尼西亚人。

问题：2005 至 2009 年间，哪两个国籍的人在巴林居住的人数相同？答案 [未提供]：巴基斯坦人和菲律宾人

Description of GPQA (Science) for the meta agent.

您的目标是找到一个在研究生级别谷歌验证问答基准（GPQA）上表现优异的智能体。该基准包含生物学、物理学和化学领域的多项选择题，由领域专家设计，以确保高质量和高难度。

An example question from GPQA:

能量分别为 E1 和 E2 的两个量子态，其寿命分别为 10^{-9} 秒和 10^{-8} 秒。我们希望清晰地区分这两个能级。下列哪个选项可能是它们的能量差，以便能够被清晰分辨？

Answer choices:

- 10^{-9} eV
- 10^{-8} eV
- 10^{-7} eV
- 10^{-6} eV

正确答案 [未提供]:

10^{-7} eV

解释 [未提供]: 根据不确定性原理, $\Delta E \cdot \Delta t = \hbar/2$. Δt 为寿命, ΔE 为

能级宽度。当 $\Delta t = 10^{-9} \text{ s} \Rightarrow \Delta E = 3.3 \cdot 10^{-7} \text{ 电子伏特}$ 时, $\Delta t = 10^{-11} \text{ 秒}$ 给出 $\Delta E = 3.3 \cdot 10^{-8} \text{ 电子伏特}$ 。因此, 两个态之间的能量差必须显著大于 10^{-7} 电子伏特 。所以答案是 10^{-4} ev 。

Description of MGSM (Math) for the meta agent.

您的目标是找到一个在多种语言小学数学基准 (Multilingual Grade School Math Benchmark, MGSM) 上表现优异的智能体, 该基准评估跨多种语言的数学问题解决能力, 以确保广泛且有效的多语言性能。

An example question from MGSM:

****Question**:** この数学の問題を解いてください。

近所では、ベットのウサギの数がベットの犬と猫を合わせた数よりも12匹少ない。犬1匹あたり2匹の猫がおり、犬の数は60匹だとすると、全部で近所には何匹のペットがいますか？

****Answer (Not Given)**:** 348

Description of MMLU (Mult-task) for the meta agent.

您的目标是找到一个在大规模多任务语言理解 (MMLU) 基准上表现优异的智能体, 该基准是一项具有挑战性的评估, 用于衡量模型在广泛学科和难度级别上回答问题的能力, 涵盖科学、技术、工程、数学 (STEM)、社会科学、人文学科等领域。

An example question from MMLU:

Answer the following multiple-choice question.

The constellation ... is a bright W-shaped constellation in the northern sky.

- (A) Centaurus
- (B) Cygnus
- (C) Cassiopeia
- (D) Cepheus

F 基线

在本文中, 我们实现了五个最先进的手工设计智能体基线, 用于 ARC 实验 (第 4.1 节): (1) 思维链 (Chain-of-Thought, COT) (Wei 等, 2022), (2) 带思维链的自治性 (Self-Consistency with Chain-of-Thought, COT-SC) (Wang 等, 2023b), (3) 自我精炼 (Self-Refine) (Madaan 等, 2024; Shinn 等, 2023), (4) 大语言模型辩论 (LLM-Debate) (Du 等, 2023), 以及 (5) 质量多样性 (Quality-Diversity), 这是智能 Go-Explore 的简化版本 (Lu 等, 2024c)。

除了这些基线之外, 我们还为推理和问题解决领域的实验 (第 4.2 节) 实现了另外两个基线: (6) 回溯抽象 (Step-back Abstraction) (Zheng 等, 2023) 和 (7) 角色分配 (Role Assignment) (Xu 等, 2023)。使用我们的简单框架实现自我精炼的示例见附录 C。

在思维链 (COT) 中, 我们提示基础模型在回答问题之前逐步思考。在带思维链的自治性 (COT-SC) 中, 我们采样 $N = 5$ 个答案, 然后使用多数投票或基础模型查询进行集成。在自我精炼 (Self-Refine) 中, 我们允许最多五次精炼迭代, 如果评论者认为答案正确则提前停止。在大语言模型辩论 (LLM-Debate) 中, 每个辩论模块被分配一个独特的角色, 例如物理专家或化学专家, 辩论持续两轮。在质量多样性 (Quality-Diversity) 中, 我们进行三次迭代, 以基于先前提出的答案收集多样化的答案。在角色分配 (Role Assignment) 中, 我们首先使用基础模型查询从预定义集合中选择一个角色, 然后使用另一个基础模型查询通过在该角色内行动来回答问题。

G 示例智能体

在本节中，我们展示了由图 1 中的元智能体搜索发现的三个示例智能体的详细实现。“多步同行评审智能体”和“分而治之智能体”是在阅读理解领域（GPQA）（Rein 等，2023）的搜索过程中发现的，而“验证多模态智能体”是在数学领域（MGSM）（Shi 等，2023）的搜索过程中发现的。

代码 4: 示例发现的智能体：多步同行评审智能体

```
def forward(self, taskInfo):
    initial_instruction = "请逐步思考，然后解决任务。"
    critique_instruction = "请审阅上述答案，并就其可能出错之处提供反馈。如果你绝对确定它是正确的，请在'correct'中输出'True'。"
    refine_instruction = "鉴于之前的尝试和反馈，仔细考虑你在最近一次尝试中可能出错的地方。利用之前尝试的洞见，尝试更好地解决任务。"
    final_decision_instruction = "鉴于以上所有思考和答案，仔细推理它们并给出最终答案。"
```

```
FM_modules = [FM_module(['thinking', '答案'], '功能模块', role=角色) 对于角色在 ['物理专家', '化学专家', '生物专家', '科学通才']]
critic_modules = [FM_module(['feedback', '正确'], '批评者', role=角色) 对于角色在 ['物理批评者', '化学批评者', '生物批评者', '通用批评者']]
final_decision_module = FM_module(['thinking', '答案'], '最终决策', temperature=0.1)
```

```
all_thinking = [] 对于 _ 在 range(len(FM_modules))
all_answer = [] 对于 _ 在 range(len(FM_modules))
all_feedback = [] 对于 _ 在 range(len(FM_modules))
```

```
对于 i 在 range(len(FM_modules)):
    思考, 答案 = FM_modules[i](taskInfo, initial_instruction)
    all_thinking[i].append(thinking)
    all_answer[i].append(answer)
```

```
对于 i 在 range(len(FM_modules)):
    对于 j 在 range(len(FM_modules)):
        if i != j:
            反馈, 正确 = critic_modules[j](taskInfo, 所有思考 [i][0], 所有答案 [i][0], critique_instruction)
            all_feedback[i].append(feedback)
```

```
对于 i 在 range(len(FM_modules)):
    refine_inputs = [taskInfo, 所有思考 [i][0], 所有答案 [i][0]] + all_feedback[i]
    思考, 答案 = FM_modules[i](refine_inputs, refine_instruction)
    all_thinking[i].append(thinking)
    all_answer[i].append(answer)
```

```
final_inputs = [taskInfo] + [所有思考 [i][1] 对于 i 在 range(len(FM_modules))] + [所有答案 [i][1] 对于 i 在 range(len(FM_modules))]
思考, 答案 = final_decision_module(final_inputs, final_decision_instruction)
```

```
返回 答案
```

代码 5: 示例发现的智能体: 分而治之的智能体 def forward(self, taskInfo): # 步骤 1: 将问题分解为子问题 decomposition_instruction = "请将问题分解为更小的、可管理的子问题。清楚地列出每个子问题。" decomposition_module = FM_Module(["思考", "子问题"], '分解模块')

步骤 2: 将每个子问题分配给专门的专家 sub_problem_instruction = "请逐步思考, 然后解决子问题。" specialized_experts = [FM_Module(["思考", "子解决方案"], '专门专家', role=role) 对于角色在 ['物理专家', '化学专家', '生物专家', '通用专家']

步骤 3: 将子问题的解整合为最终答案 integration_instruction = "给定子问题的解, 将它们整合起来, 为原始问题提供最终答案。" integration_module = FM_Module(['thinking', '答案'], '整合模块', temperature=0.1)

分解问题思考, sub_problems = decomposition_module([任务信息], decomposition_instruction)

确保 sub_problems 是字符串并拆分为单个子问题 sub_problems_list = 子问题.内容.拆分 ('\\n') 如果是 sub_problems.content,字符串) 否则 []

解决每个子问题 sub_solutions = [] for i, sub_problem in enumerate(sub_problems_list): sub_problem_info = Info('sub_problem', decomposition_module. __repr__(), sub_problem, i) sub_thinking, sub_solution = specialized_experts[i % len(specialized_experts)]([sub_problem_info], sub_problem_instruction) sub_solutions.append(sub_solution)

整合子问题解决方案 integration_inputs = [任务信息] + sub_solutions 思考, 回答 = integration_module(integration_inputs, integration_instruction)

返回 答案

代码 6: 示例发现的智能体: 验证多模态智能体 def forward(self, taskInfo): # 生成问题视觉表示的指令 visual_instruction = "请创建给定问题的视觉表示 (例如, 图表、图形)。"

验证视觉表示的指令 verification_instruction = "请验证视觉表示的准确性和相关性。如有必要, 请提供反馈和改进建议。"

使用已验证的视觉辅助工具解决问题的指令 cot_instruction = "利用所提供的视觉表示, 逐步思考并解决问题。"

```

# 实例化视觉表示模块、验证模块和思维链模块 visual_module = FM_Module(["视觉的"], '视觉表示模块')
verification_module = FM_Module(["反馈", "已验证的视觉"], '验证模块') cot_module = FM_Module(['thinking', '答案'], '思维链模块')

# 生成问题的视觉表示 visual_output = visual_module([任务信息], visual_instruction)
visual_representation = 视觉输出 [0] # 直接使用信息对象

# 验证视觉表示反馈, verified_visual = verification_module([任务信息, 可视化表示], verification_instruction)

# 使用已验证的视觉表示来解决问题 思考, 回答 = cot_module([任务信息, 已验证视觉], cot_instruction) 返回答案

```

H 元代理搜索的伪代码

在本节中，我们提供元代理搜索算法的伪代码，以阐明其实现和 workflows。该伪代码概述了使用元代理设计、评估和优化代理的迭代过程，如正文所述。

Algorithm 1 Meta Agent Search Algorithm

```

1: Input: Target domain validation data, maximum iterations  $N$ 
2: Output: Archive of discovered agents
3: Initialize archive  $\mathcal{A}$  with baseline agents (e.g., Chain-of-Thought, Self-Refine)
4: for  $i = 1$  to  $N$  do
5:   Design Step: Meta agent generates a new agent:
6:     (a) Outputs design reasoning
7:     (b) Implements the design in code
8:     (c) Performs two self-reflection steps to ensure novelty and correctness
9:   Evaluation Step: Evaluate the new agent on target domain validation data:
10:    (a) If the agent produces errors during evaluation, refine the design up to 5 iterations
11:    (b) Re-run the evaluation after each refinement
12:   Update Step: Add the refined agent and its evaluation metrics to the archive  $\mathcal{A}$ 
13: end for
14: Return: Final archive  $\mathcal{A}$ 

```

I 初始化的影响

本文的核心主张之一是代码空间表示能够更好地利用现有的人类成果（第 2 节），从而比完全从零开始实现更高效的搜索过程。为了进一步研究初始化的影响，我们进行了实验，在不提供任何初始智能体设计的情况下运行元智能体搜索算法，并与将人类设计的解决方案纳入搜索过程的标准方法进行对比。

表 6 中呈现的结果表明，即使没有初始智能体设计，元智能体搜索在所有评估领域中都发现了优于所有手工设计基线的智能体。这一发现凸显了本文方法的鲁棒性，因为它有效地利用了代码空间的内在结构来探索和优化智能体设计。

有趣的是，虽然纳入良好的初始解决方案通常会带来性能提升，但数学领域却出现了独特的结果：从零开始反而获得了更优的性能。我们假设，在这种情况下，缺乏预定义的设计模式鼓励了在有限迭代次数内对推理策略进行更广泛和更多样化的探索。这种多样性对于需要灵活多变推理方法的数学任务似乎特别有益。

这一观察为未来研究开辟了一个有趣的途径：探索初始化的选择和初始解的质量如何影响不同领域中的搜索有效性。例如，确定在哪些条件下从无初始解开始可能带来性能提升，或者设计结合初始化和广泛探索两者优势的策略，都将是有价值的研究方向。

表 6：元智能体搜索在多个领域中包含和不包含初始智能体设计的性能比较。结果表明，即使没有初始化，元智能体搜索在所有领域中都优于手工设计的基线。然而，纳入初始解决方案通常会带来更好的性能，但数学领域除外，在该领域中从无初始化开始反而获得了更优的结果。

Agent Name	F1 Score	Accuracy (%)		
	Reading Comprehension	Math	Multi-task	Science
State-of-the-art Hand-designed Agents				
Chain-of-Thought (Wei et al., 2022)	64.2 ± 0.9	28.0 ± 3.1	65.4 ± 3.3	29.2 ± 3.1
COT-SC (Wang et al., 2023b)	64.4 ± 0.8	28.2 ± 3.1	65.9 ± 3.2	30.5 ± 3.2
Self-Refine (Madaan et al., 2024)	59.2 ± 0.9	27.5 ± 3.1	63.5 ± 3.4	31.6 ± 3.2
LLM Debate (Du et al., 2023)	60.6 ± 0.9	39.0 ± 3.4	65.6 ± 3.3	31.4 ± 3.2
Step-back Abstraction (Zheng et al., 2023)	60.4 ± 1.0	31.1 ± 3.2	65.1 ± 3.3	26.9 ± 3.0
Quality-Diversity (Lu et al., 2024c)	61.8 ± 0.9	23.8 ± 3.0	65.1 ± 3.3	30.2 ± 3.1
Role Assignment (Xu et al., 2023)	65.8 ± 0.9	30.1 ± 3.2	64.5 ± 3.3	31.1 ± 3.1
Automated Design of Agentic Systems on Different Domains				
Meta Agent Search (Empty Initialization)	73.9 ± 0.9	67.5 ± 3.3	68.5 ± 3.3	32.7 ± 3.2
Meta Agent Search	79.4 ± 0.8	53.4 ± 3.5	69.6 ± 3.2	34.6 ± 3.2

J 实验成本

在 ARC（第 4.1 节）上进行一次搜索和评估的运行成本约为 500 美元（OpenAI 应用程序编程接口费用），而在推理和问题解决领域（第 4.2 节）中的一次运行成本约为 300 美元。

主要开销来源于在评估所发现智能体时查询“gpt-3.5-turbo-0125”模型。值得注意的是，最新的 GPT-4 模型“gpt-4o-mini”的价格不到“gpt-3.5-turbo-0125”的三分之一，且性能更优，这表明我们能够以仅三分之一的成本通过元智能体搜索获得更佳结果。此外，如第 6 节所述，当前朴素的评估函数既昂贵又忽略了有价值的信息。我们预计，未来工作采用更精细的评估函数可显著降低 ADAS 算法的成本。